

Exponent Vaults Audit Extension

Table of Contents

1. Executive Summary

5

About Exponent

5

Audit Summary

5

Overall Security Posture

6

Before Fix Review Summary

6

After Fix Review Summary

7

Audit Scope

7

2. Assumptions and Considerations

8

Audit Assumptions

8

3. Severity Definitions

9

Impact

9

Likelihood

9

Severity Classification Matrix

9

4. Findings

11

C01: Omitting the Kamino delegated-farm user_state from remaining_accounts silently deflates vault AUM, letting any user mint LP at a discount

11

H01: A 1-lamport donation to a closed Loopscale loan PDA after external liquidation bricks every user-facing AUM-recalc flow with no in-program recovery

13

M01: A manager can bleed vault value through adverse-price swaps routed via a pool they control, since the swap validators do not bound realized output

17

M02: A manager can drain vault funds by staking into a self-created Kamino farm, since staking auto-tracks the farm with no governance or legitimacy check

19

M03: A manager can drain vault funds by registering a Kamino obligation in a self-created lending market

21

M04: Kamino farm reward mints are never checked for priceability, an unpriceable reward freezes recalc for the whole vault

24

M05: Several interaction validators compute aum_diff without classifying the source/destination ATA's bucket (reserve vs position), producing values that diverge from the real change to aum_in_base_in_positions	26
M06: Kamino Lending validator does not constrain the user-side liquidity ATA in deposit/withdraw/borrow/repay, letting the manager hide and restore tracked AUM to dilute LP holders	30
M07: wrapper_remove_offer validator skips destination tracking, letting the manager redirect the cancelled offer's PT / YT / SY payload to any token account	33
M08: Loopscale CreateLoan validator does not bind borrower to the vault, letting the manager track a loan whose collateral they control outside the vault and freely move the LP share price	36
M09: Loopscale CreateStrategy validator does not bind params.lender to the vault, letting the manager track a strategy whose balance they control outside the vault and freely move the LP share price	39
L01: remove_token_entry's reference guard only checks orderbook positions, so removing an entry still priced by a non-orderbook position blocks all vault deposits and withdrawals	42
L02: Kamino-vault share price has no staleness bound, so deposits and withdrawals can be priced against a lagged AUM	45
L03: Kamino Lending collateral-withdraw validator trusts caller-declared collateral_amount, which Kamino silently caps to deposited collateral, letting the manager bypass the reserves invariant	48
L04: A positions snapshot that is stale across transactions will let a manager deploy past the reserves cap	51
L05: Borrow validators let borrowed funds land in the same account that holds the vault's deposit-funded reserves, making the two indistinguishable for fast-withdrawal and cap accounting	54
L06: update_price on a TWAP entry delays the next update_twap_price, slowing TWAP observations below the configured cadence	57
L07: Kamino Farms stake validator reads the source token balance after the inner CPI has already drained it, producing a wrong aum_diff for the u64::MAX "stake everything" path	60
L08: Loopscale withdraw_strategy and withdraw_collateral validators trust the caller-declared amount even when withdraw_all=true silently caps the actual transfer, letting the manager bypass the reserves invariant	63
L09: Kamino Vault withdraw validator trusts the caller-declared shares_amount even when Kamino silently caps to user_shares_ata.amount, letting the manager bypass the reserves invariant	67

5. Enhancement Opportunities70

E01: Redundant manual recount rollback before return Err in price-chain registration handlers	70
---	----

E02: Validating the TWAP window size against the minimum the stored-price path requires will fail misconfiguration early instead of as a vault-wide recalc revert72

About Us74

About Adevar Labs74

Audit Methodology74

Confidentiality Notice76

Legal Disclaimer76

1. Executive Summary

About Exponent

Exponent is a Solana yield protocol built around principal-token (PT) and yield-token (YT) splits of yield-bearing assets, with an integrated orderbook for PT/YT trading. This engagement is an extension of the original Exponent Vaults audit, covering the managed-vault layer that lets a vault manager deploy LP-deposited capital into a fixed set of external venues (Kamino lending and vaults, Kamino farms, Loopscale loans and strategies, Titan swap routing, and Exponent's own PT/YT orderbook) while tracking on-chain assets-under-management, enforcing reserve floors for fast LP withdrawals, and pricing LP shares against a recalculated AUM.

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	1	1	0
High	1	1	0
Medium	9	9	0
Low	9	7	2

Enhancement opportunities: 2 (both implemented)

Overall Security Posture

The managed-vault layer uses a defense-in-depth model: a Squads multisig gates manager actions, a post-hook validates every integrated-program interaction, LP governance with timelocks controls privileged configuration, and a sentinel role acts as a final safety net. Within that model, the access required to exercise a finding matters as much as its technical impact. The overwhelming majority of the issues found require a malicious or compromised privileged actor – the vault manager, an admin, or a governance proposal – to exploit; they are reported because the validators should defend in depth even against a misbehaving privileged party, not because an arbitrary user can trigger them. Permissionless exposure is limited to the Critical AUM-dilution issue and the loan-PDA griefing brick. A further permissionless issue on a legacy migration path is treated as out of scope of the deployed surface and recorded in the assumptions section. The audit found the issues primarily at the boundaries between the layers: account-binding and ownership linkage on externally-registered positions, cross-instruction state aggregation in the hook, and amount and price handling in AUM accounting. The Exponent team engaged closely on every finding; the volume reflects integration breadth, not gaps in the team's protocol understanding.

Before Fix Review Summary

The audit identified one Critical, one High, nine Medium, and nine Low severity findings, plus two enhancement opportunities.

The two permissionless findings are the most directly user-facing. The Critical finding is that omitting a Kamino delegated-farm **user_state** account from **remaining_accounts** silently under-counts AUM by the pending reward value, letting the next depositor mint LP shares at a discount. The High finding is a 1-lamport donation to a closed Loopscale loan PDA after external liquidation that bricks every user-facing AUM-recalc flow with no in-program recovery.

The remaining findings require privileged access. The nine Medium findings concentrate on validator correctness against a misbehaving manager: account-binding and legitimacy gaps on externally-registered Loopscale loans and strategies and on self-created Kamino farms and lending markets, the orderbook-offer cancel validator that skipped destination tracking, the Kamino Lending validator that did not constrain the user-side liquidity ATA, a validator family that computed **aum_diff** without classifying the source or destination bucket, unpriceable Kamino farm rewards freezing recalc, and adverse-price swap routing through a manager-controlled pool. The nine Low findings are manager-gated paths that do not move funds out of vault control: several reserve-invariant bypasses where a validator trusted a caller-declared amount the external program silently caps (Kamino Vault withdraw, Loopscale withdraw-all, Kamino Lending collateral-withdraw), the Kamino Farms **stake** post-CPI balance read, the cross-transaction stale positions snapshot, the borrow-window reserve commingling, the Kamino-vault share-price staleness bound, the **remove_token_entry** reference guard, and **update_price** delaying the next **update_twap_price**. The two enhancements cover validating the TWAP window size against the stored-price minimum and removing a redundant manual refcount rollback. One further issue, a permissionless price-chain pin on the legacy **migrate_strategy_sy_prices** path, is treated as out of scope of the deployed surface and recorded in the assumptions section.

After Fix Review Summary

Of the twenty-two in-scope findings, twenty are fixed (including both enhancements), and two are acknowledged; a further legacy-migration issue is recorded in the assumptions section as out of scope.

The team made an architectural change that supersedes a whole class of the validator findings: the per-instruction `aum_diff` machinery was removed and replaced with end-of-transaction final-balance accounting in the hook. Validators now perform only account-binding checks and emit Track/Untrack mutations, so they no longer trust caller-declared amounts at all, closing the silent-cap and bucket-classification issues at the root. Both permissionless findings are fixed: the Critical `user_state` omission now errors with `MissingAccount`, and the closed-loan brick fix no longer keys on lamport balance, with the new direct-manager removal path gated on the loan being actually closed. Cross-transaction staleness is closed by a refresh-required flag plus a new permissionless `refresh_aum`, backed by a per-vault-configurable NAV circuit breaker.

A regression introduced during the refactor was caught and resolved: the orderbook-offer cancel destination tracking (`validate_wrapper_exit_trader_accounts_tracked`) had been removed, and after it was flagged the team reintroduced it in a follow-up commit to PR 3894, so a cancelled offer's payload can no longer be redirected to an untracked account. The adverse-price swap guard's slippage bound, initially a 5000 bps placeholder, was tightened to 350 bps following the review.

Two findings are acknowledged: the borrow-window reserve commingling, which the team confirmed is intended design (LPs filling withdrawals from reserves that may hold borrowed funds is accepted), and the Kamino-vault share-price staleness bound (pending a refresh primitive from Kamino). An additional change submitted on top of the remediation (PR 3908, allowing active-vote unstaking during the open voting window in `unstake_vote`) was reviewed and introduced no new findings. The finding classifications reflect the final severities agreed with the team following the fix review.

Audit Scope

- **Engagement:** Extension of the original Exponent Vaults audit.
- **Repository:** <https://github.com/exponent-finance/exponent-monorepo>
- **Before-Fix Review Commit Hash:** `fb4870a117d7722f35acb053b051538f925811d9`
- **After-Fix Review Commit Hash:** `bdf05881307bff1ce59be18f4413a93d5279283d`
- **Files/Modules in Scope:**
 - `solana/libraries/exponent_vaults_integrations/src/*.rs`

2. Assumptions and Considerations

Audit Assumptions

I. Manager acts in good faith. The vault manager is assumed to operate the vault in LP holders' interests, gated by off-chain governance around the Squads multisig. Findings that require a malicious manager (Loopscale **CreateLoan borrower** binding, adverse-price swap routing, self-created Kamino farms and lending obligations, self-created Loopscale loans and strategies) are reported as defense-in-depth gaps and rated under this baseline.

II. External programs behave per their documented interfaces. Kamino Lending, Kamino Vaults, Kamino Farms, Loopscale, Titan, and the Exponent PT/YT orderbook are not modeled as adversarial. Silent-cap behavior on caller-declared amounts (Kamino **collateral_amount**, **shares_amount**, and **liquidity_amount**; Loopscale **withdraw_all = true**; the Kamino Farms **u64::MAX** stake sentinel resolving against the pre-CPI **user_ata.amount**) is treated as documented behavior the vault validators must defend against.

III. Price publishers are accurate within their published staleness. TWAP entries, stored-price chains, and Kamino-vault share-price reads are trusted to publish accurate values inside their configured windows. The vault is responsible for enforcing its own per-vault staleness bounds.

IV. Audited code matches the reference mirror at commit

fb4870a117d7722f35acb053b051538f925811d9. Any divergence between that commit and the code deployed to mainnet is out of scope for this report.

V. The legacy **migrate_strategy_sy_prices path is out of scope of the deployed surface.** This one-shot migration handler filled **price_id_sy** on legacy **OrderbookEntry** and **YieldPositionEntry** strategy positions and is not reachable on the deployed vaults; the remediation removed it entirely. It is treated as out of scope and preserved here for completeness. Were it in scope, the permissionless, signer-unchecked handler would let the first caller, rather than the manager, pin which price chain a legacy strategy position resolves against, since it accepts the caller-supplied **price_id_sy** and validates only the chain shape.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

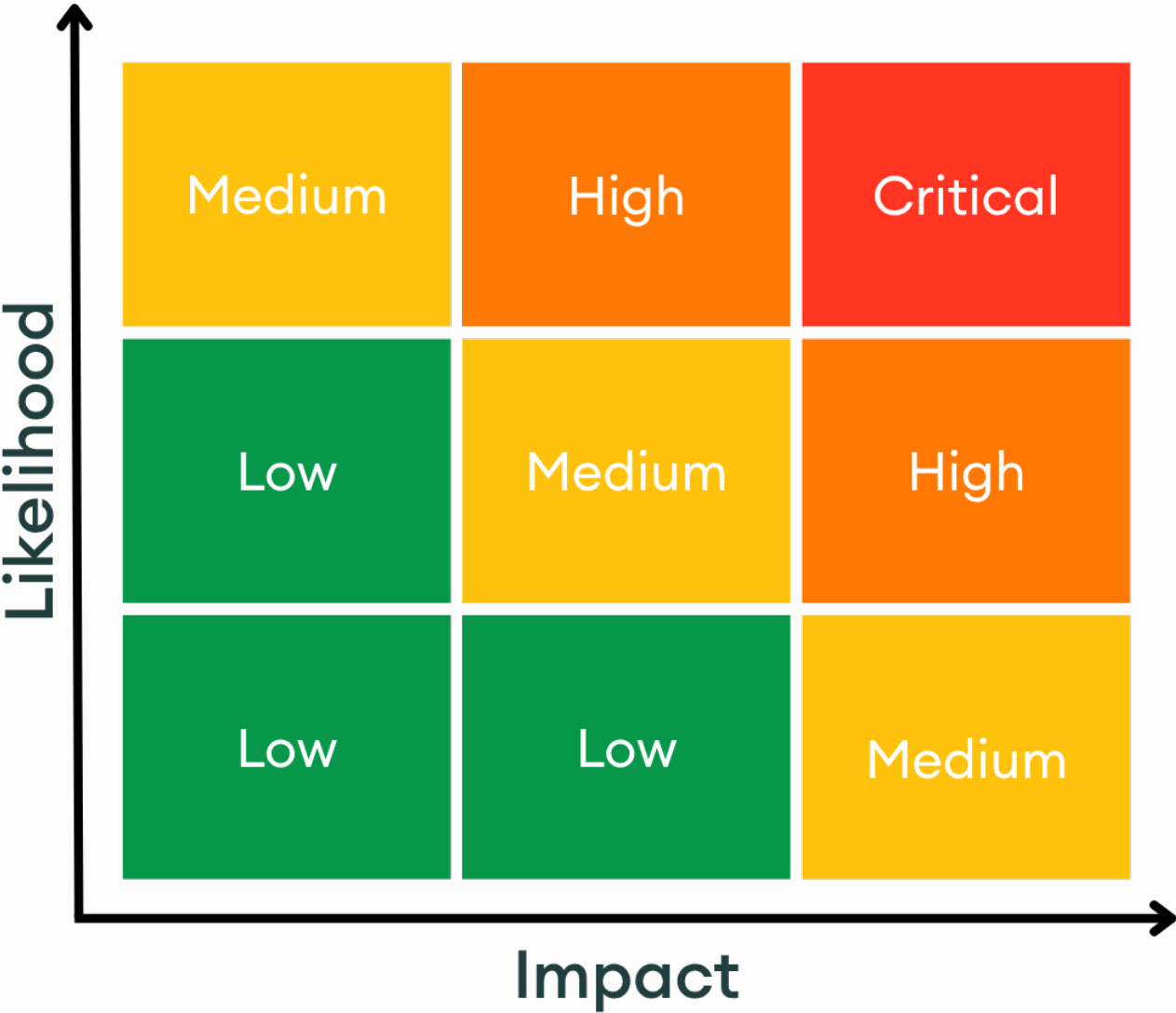
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability

- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

C01: Omitting the Kamino delegated-farm **user_state** from **remaining_accounts** silently deflates vault AUM, letting any user mint LP at a discount



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/position_values/kamino_lend.rs:L303-L308 [↗](#)

RELEVANT SNIPPET

```
>_kamino_lend.rs RUST
301: // reserve. In that state there is no reward-bearing user account to
302: // project, so delegated reward contribution is intentionally zero.
303: let Some(user_account) = remaining_accounts
304:     .iter()
305:     .find(|account| account.key == &user_state)
306: else {
307:     continue;
308: };
309:
310: let user = read_user_state(user_account)?;
```

DESCRIPTION

calculate_delegated_farm_reward_aum_in_base looks up each delegated farm's **user_state** PDA in the caller-supplied **remaining_accounts**. If the lookup returns **None**, the function silently skips that farm's pending reward contribution:

```
RUST
let Some(user_account) = remaining_accounts
    .iter()
    .find(|account| account.key == &user_state)
else {
    continue;
};
```

Every other account this code path consults uses **find_account** or **find_required_account**, both of which return an error if the account is missing. The **user_state** lookup is the only one that proceeds

when the account is absent, so a caller can omit only this one account to under-count AUM by exactly that farm's pending reward value, while every other lookup still succeeds and the recalc completes normally.

Because `recalculate_aum_in_base` is the source of truth for LP share-price in `lp_out_from_base` and `base_out_from_lp`, deflating it on a deposit causes the depositor to receive more LP per token than they should. They can then withdraw against the restored AUM (with the `user_state` included) and walk away with the difference, paid out of existing LP holders' share value.

The `reserve_farm_mappings` that drive this loop are populated automatically at obligation registration time by `derive_reserve_farm_mappings`, reading each reserve's live `farm_collateral` and `farm_debt`. There is no add or remove API for delegated farms specifically. Any vault whose tracked Kamino obligation touches a klend reserve carrying a delegated farm (which is the standard mechanism Kamino uses to distribute incentive rewards) is exposed.

PROOF OF CONCEPT

Setup: vault has a tracked Kamino obligation, the obligation's `reserve_farm_mappings` include at least one delegated farm, and that farm has accrued some pending reward AUM R . Total real AUM is X .

1. Attacker calls `deposit_liquidity(Z)` and supplies `remaining_accounts` with the obligation, every reserve, every farm `farm_state`, every `global_config`, and every `scope_prices`, but omits the farm `user_state` PDA. Recalc skips that farm's reward via the silent `continue`, returning visible AUM of $X - R$. Attacker is minted LP at this deflated share-price, receiving more LP than they would have at the true AUM X .

2. Attacker calls `execute_withdrawal_from_reserves` with the bonus LP and now passes the `user_state` in `remaining_accounts`. Recalc returns the full $X + Z$. Attacker burns the LP at the restored share-price and receives more tokens than they deposited, less the standard withdrawal-fee bps.

The difference is paid out of existing LP holders. Repeatable until rewards are harvested.

RECOMMENDATION

Make the `user_state` account required in `remaining_accounts`, the same way every other input to this function is. The original comment justifies the silent skip with "klend may not have initialized the delegated farm user yet," but that case can be handled by inspecting the account's data after a required lookup (treat empty or wrong-discriminator as zero rewards) rather than by allowing the account to be missing. The caller then cannot decide which farms get counted.

FIX REVIEW NOTES

Fixed in PR 3878. The validator now requires the Kamino delegated-farm `user_state` PDA in `remaining_accounts`; omission returns `MissingAccount` instead of silently zeroing the delegated-farm reward, so vault AUM can no longer be under-counted by the pending reward value.

H01: A 1-lamport donation to a closed Loopscale loan PDA after external liquidation bricks every user-facing AUM-recalc flow with no in-program recovery



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/position_values/loopscale.rs:L30-L34 [↗](#)

RELEVANT SNIPPET

```
>_loopscale.rs RUST
28:  /// closed-shape; the stricter predicate causes the caller's live-account path to
29:  /// fail loud on that anomalous case rather than silently classifying it as closed.
30:  pub fn is_account_closed(account: &AccountInfo) -> bool {
31:      account.data_is_empty()
32:      && account.lamports() == 0
33:      && account.owner == &anchor_lang::system_program::ID
34:  }
35:
36:  /// Loopscale's `interest_per_second` (PodDecimal) encodes a per-second token-base-unit
```

DESCRIPTION

When a Loopscale liquidator (an **LIQ_AUTHS**-allowlisted signer, not the Exponent manager) calls **liquidate_ledger** directly against a vault-owned loan, the handler zeroes the affected ledger (sets status to **Uninitialized**) and seizes the collateral assigned to that ledger (sets each affected collateral slot's **asset_mint** back to the default zero pubkey). At the end of the handler, Loopscale checks **is_eligible_for_close()**, which returns true when every ledger is inactive AND every collateral slot is zeroed AND the loan is not locked. When all three hold, the handler closes the loan account in the same transaction. In the Loopscale repo, [programs/loopscale/src/instructions/creditbook/liquidate.rs](#):

```
RUST
if ctx.accounts.loan.load()?.is_eligible_for_close() {
    msg!("Closing loan");
    ...
    ctx.accounts.loan.close(ctx.accounts.payer.to_account_info());
}
```

For a loan with one active ledger and only that ledger's mapped collateral, a single liquidation drives all three conditions to true and the loan closes. For multi-ledger loans, the same outcome follows from sequential liquidations across every ledger.

This close happens entirely outside the Exponent program. No Squads sync, no `validate_interaction_hook`, no `UntrackLoopscaleLoan` mutation. The vault still tracks the now-closed loan in `vault.strategy_positions`. The closed PDA has `data_is_empty()`, `lamports() == 0`, `owner == system_program::ID`.

The only in-program mechanism that removes a stale tracked Loopscale loan is `CleanupResult::LoopscaleLoanRemove { remove_position: true }`, emitted by `get_aum_loopscale_loan (aum.rs:296-303)` when `is_account_closed(loan_account)` returns true. The predicate strictly requires `lamports() == 0 (position_values/loopscale.rs:40-44)`.

Anyone can `system_program::transfer { to = <closed_loan_pda>, lamports = 1 }`. After the donation the PDA holds 1 lamport while data and owner stay system-closed.

The next call to `calculate_loan_aum_in_base` runs this gate again:

RUST

```
if is_account_closed(loan_account) {
    return Ok(0);
}
require_keys_eq!(
    *loan_account.owner,
    LOOPSCALE_PROGRAM_ID,
    ErrorCodeVaults::InvalidAccountData
);
```

`is_account_closed` returns false (`lamports != 0`), the owner check fails (owner is `system_program`, not Loopscale), and `recalculate_aum_in_base` reverts.

Every flow that calls `recalculate_aum_in_base` is now bricked: deposits, withdrawals (fast and normal), manager fills, and the wrapper-based manager actions. Queued withdrawals stay locked, the filled portion stranded in `token_account_vault`. The donated lamport cannot be drained because system-owned PDAs have no signer.

The tracked entry cannot be removed by any in-program path either: **neither `PositionUpdate` nor `VaultSettingsAction`** includes a Loopscale-remove variant, and the validator's `UntrackLoopscaleLoan` mutation only fires when a Loopscale CPI (`LoopscaleInstructionKind::CloseLoan`) through Squads runs against the loan, which now fails Anchor's owner check (because the loan is already closed by the liquidator). Recovery requires a program upgrade.

PROOF OF CONCEPT

Pre-state: a vault using Loopscale as a borrower (a tracked `LoopscaleLoan` registered via Squads `create_loan`) whose ledgers and collateral are configured such that a complete liquidation drives the loan to close-eligible (the typical case: one active ledger backed by the collateral mapped to that ledger).

1. The vault's borrower position decays past Loopscale's liquidation threshold (price movement or interest accrual).
2. An `LIQ_AUTHS`-allowlisted liquidator submits `liquidate_ledger` directly to Loopscale. The handler transfers the principal repayment to the strategy, transfers the collateral to the liquidator, zeroes the

ledger and the affected collateral slots, then evaluates `is_eligible_for_close()` (now true) and calls `loan.close(payer)`. After this tx: loan PDA has `data=[]`, `lamports=0`, `owner=system_program::ID`. The vault is unaware and still tracks the loan entry.

3. Attacker submits `system_program::transfer { from = attacker, to = <closed_loan_pda>, lamports = 1 }`. After this tx: loan PDA has `data=[]`, `lamports=1`, `owner=system_program::ID`.

4. Any user calls `deposit_liquidity` and passes the closed PDA in `remaining_accounts` (they have to, otherwise `find_account` errors with `MissingAccount`). The handler invokes `vault.recalculate_aum_in_base`, which iterates `strategy_positions`, hits the Loopscale loan, calls `get_aum_loopscale_loan`. `is_account_closed` returns false (`lamports != 0`). Falls through to `calculate_loan_aum_in_base`, where `require_keys_eq!(*loan_account.owner, LOOPSCALE_PROGRAM_ID, ErrorCodeVaults::InvalidAccountData)` fails. Recalc reverts. The same failure mode applies to every other recalc-dependent user flow.

5. Attempted recovery, every path fails:

- Manager calls Squads `close_loan`: Anchor's `AccountLoader<'info, Loan>` rejects the system-owned account; inner ix reverts, validator's `UntrackLoopscaleLoan` mutation never fires.
- Manager / sentinel calls `wrapper_manager_update_position` or `manager_update_position`: `PositionUpdate` enum has no Loopscale-remove variant.
- Manager / sentinel routes through a governance proposal: `ProposalAction::PositionUpdate` and `VaultSettingsAction` also lack Loopscale-remove variants.
- Manager tries Loopscale's `create_loan` with the same `(borrower, nonce)` to put a fresh loan at the same PDA: Anchor `init` requires zero lamports on the destination, so the donated lamport blocks it. Unpredictable nonces normally prevent pre-funding squatting, but the nonce here is already on-chain from the original creation, so randomness does not help in this scenario.
- Anyone tries to drain the donated lamport: system-owned PDA has no signer, so impossible.

Working flows after the attack are limited to `queue_withdrawal` (escrows LP but the filled portion can never be executed), `cancel_withdrawal` (only if no fills are present), `collect_management_fee`, sentinel flag-setting, and voting. The donation persists until a program upgrade.

RECOMMENDATION

Soften `is_account_closed` to match Anchor's own `is_closed` predicate by dropping the `lamports() == 0` clause:

```
pub fn is_account_closed(account: &AccountInfo) -> bool {
    account.data_is_empty() && account.owner == &anchor_lang::system_program::ID
}
```

RUST

The existing comment justifies the strict clause as "fail loud on the anomalous case", but the failure has no in-program operator fix, so the outcome is silent, permanent breakage. `data_is_empty() + owner == system_program` is sufficient on its own: a 1-lamport donation cannot inject phantom AUM because there is no data to read, and an attacker cannot pre-position data at a not-yet-created PDA address (writing to a PDA needs a program that knows the seeds).

As a defense-in-depth measure, add explicit `RemoveLoopscaleLoan { loan: Pubkey }` and `RemoveLoopscaleStrategy { strategy: Pubkey }` variants to `PositionUpdate` (gated to manager / allocator, or to proposal-mediated execution). These give the manager / governance an explicit

recovery path for any "stuck Loopscale entry" failure mode, including ones not yet identified, without needing a program upgrade.

FIX REVIEW NOTES

Fixed in PR 3885. `is_account_closed` no longer keys on `lamports == 0`, so a lamport donation no longer flips the closed-loan classification. The follow-up in PR 3900 gates the new direct-manager `RemoveLoopscaleLoan` and `RemoveLoopscaleStrategy` actions on `is_account_closed(loan_account)`, so a manager can untrack only an actually-closed position rather than any live one.

M01: A manager can bleed vault value through adverse-price swaps routed via a pool they control, since the swap validators do not bound realized output



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validations/titan.rs:L39-L72 [↗](#)

RELEVANT SNIPPET

```
>_ titan.rs RUST
37: }
38:
39: /// Validate a Titan SwapRouteV2 instruction.
40: ///
41: /// Checks that input and output token accounts are vault-owned (tracked).
42: fn validate_titan_swap(
43:     vault: &VaultValidationView<'_>,
44:     _program_id: &Pubkey,
45:     data: &[u8],
46:     account_indexes: &[u8],
47:     accounts: &[AccountInfo],
48:     tracked: &BTreeSet<Pubkey>,
49: ) -> Result<Option<ValidationResult>> {
50:     let input_token_account_index = account_indexes
51:         .get(INPUT_TOKEN_ACCOUNT_INDEX)
52:         .map(|idx| *idx as usize)
53:         .ok_or(ErrorCodeVaults::UnsupportedInstruction)?;
54:
55:     let output_token_account_index = account_indexes
56:         .get(OUTPUT_TOKEN_ACCOUNT_INDEX)
57:         .map(|idx| *idx as usize)
58:         .ok_or(ErrorCodeVaults::UnsupportedInstruction)?;
59:
60:     validate_tracked_account_index(input_token_account_index, accounts, tracked)?;
61:     validate_tracked_account_index(output_token_account_index, accounts, tracked)?;
62:
63:     let input_token_amount = deserialize_u64_at(data, DISCRIMINATOR_LEN)?;
64:     let aum_diff = calculate_reserve_to_position_aum_diff_for_known_amount(
65:         vault,
66:         accounts[input_token_account_index].key,
67:         accounts[output_token_account_index].key,
68:         input_token_amount,
69:         KnownAmountSide::Source,
```


>_ titan.rs

RUST

```
70:         )?;  
71:         Ok(Some(ValidationResult::new(aum_diff, Vec::new())))  
72:     }
```

DESCRIPTION

When the manager executes a swap through the vault (a Titan **swap_route_v2**, or an **exponent_clmm_trade_pt**, **buy_yt**, or **sell_yt**), the corresponding validator confirms that the input and output token accounts are accounts the vault recognizes and records the resulting AUM change. It does not read the amount of tokens actually received, and it places no constraint on the swap route.

A malicious manager builds a swap that routes through a pool where they are the counterparty, priced so the vault spends the full input amount and receives a negligible amount of the output token. Both the input and output token accounts are vault accounts, so these checks pass and the swap is accepted. The output token lands in a vault account, while most of the input value has moved to the manager-controlled pool.

RECOMMENDATION

1. Add **max_slippage_bp** to the vault configuration. Set its initial value at vault initialization.
2. Make it a non-additive settings action, so on a live vault only the governance-proposal path can change it (the same protection the reserves share already has against direct manager changes).
3. In **validate_titan_swap**, **validate_trade_pt**, **validate_buy_yt**, and **validate_sell_yt**, read the post-execution destination-account balance delta, value it at the stored price, and require it to be at least **input_value * (BASE_POINTS - max_slippage_bp) / BASE_POINTS**.
4. Use a generous default so normal rebalancing is unaffected while a swap that loses most of the input value is rejected.

FIX REVIEW NOTES

Fixed in PR 3907. Swap validators are marked **is_swap**; the hook requires swap interactions to be isolated to a single inner instruction, and a per-vault **max_swap_slippage_bp** (configurable via governance) bounds value loss measured against initial-versus-final tracked AUM. The initial 5000 bps value was a placeholder; following the review the team lowered the default to 350 bps.

M02: A manager can drain vault funds by staking into a self-created Kamino farm, since staking auto-tracks the farm with no governance or legitimacy check



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validation
s/kamino_farms.rs:L151-L156 [↗](#)

RELEVANT SNIPPET

```
>_kamino_farms.rs RUST  
149: );  
150: let mut mutations = Vec::new();  
151: if !is_kamino_farm_tracked(vault, &farm_state_key, &user_state) {  
152:     mutations.push(VaultMutation::TrackKaminoFarm {  
153:         farm_state: farm_state_key,  
154:         user_state,  
155:     });  
156: }  
157:  
158: Ok(Some(ValidationResult::new(aum_diff, mutations)))
```

DESCRIPTION

A Kamino farm becomes an AUM-bearing tracked position as a side effect of a stake instruction. When the manager stakes vault assets into a farm through the Squads policy, **validate_stake** emits a mutation that adds the farm to the vault's tracked positions:

```
RUST  
if !is_kamino_farm_tracked(vault, &farm_state_key, &user_state) {  
    mutations.push(VaultMutation::TrackKaminoFarm { farm_state: farm_state_key, user_state });  
}
```

There is no separate registration step for standalone Kamino farms and no governance step on this path. The farm is whichever account the stake points at, identified only by Kamino program ownership (**read_farm_state** checks **account.owner == KAMINO_FARMS_PROGRAM_ID**).

Kamino farms are created permissionlessly. The farm-initialization handler in the Kamino Farms program ([programs/kfarms/src/handlers/handler_initialize_farm.rs](#)) makes the caller the farm administrator:

```
pub farm_admin: Signer<'info>,  
// ...  
farm_state.farm_admin = ctx.accounts.farm_admin.key();
```

RUST

The farm administrator sets the farm configuration and controls its lifecycle, including lockups and the conditions under which staked principal can be unstaked and withdrawn.

The manager creates a Kamino farm they administer for a mint the vault can price, then stakes vault assets into it through the Squads policy. As the farm administrator the manager sets the farm **withdraw_authority** to an address they control through **update_farm_config** (Kamino Farms program, [programs/kfarms/src/farm_operations.rs](#), **FarmConfigOption::WithdrawAuthority**), then calls **withdraw_from_farm_vault** ([programs/kfarms/src/handlers/handler_withdraw_from_farm_vault.rs](#)), which transfers staked tokens out of the farm vault to that address. The entire staked principal is taken from the vault.

RECOMMENDATION

Require governance for tracking a new Kamino farm in a vault that already holds user funds. Introduce an explicit registration step for standalone Kamino farms, separate from the stake instruction, and gate that registration so that when the vault has non-zero AUM and non-zero LP supply it is permitted only through the governance-proposal path. This matches the trust boundary the codebase already applies for live-vault setup additivity and gives LP holders the opportunity to reject or opt out of a deployment into an unvetted farm.

Permit implicit tracking from a manager stake only when the vault has zero AUM and zero LP supply, where no existing holders are exposed and future depositors can inspect the deployed positions before depositing.

FIX REVIEW NOTES

Fixed in PR 3886. Auto-tracking via **stake** was removed; a new explicit **AddKaminoFarmEntry** enforces strong checks (non-delegated farm, **user_state** PDA derived from **vault.squads_vault**, and **user.owner == delegatee == squads_vault**).

M03: A manager can drain vault funds by registering a Kamino obligation in a self-created lending market



LOCATION

exponent-monorepo/fb4870a1/solana/programs/exponent_vaults/src/instructions/wrappers/wrapper_manager_update_position.rs:L45-L59 [↗](#)

RELEVANT SNIPPET

```
>_wrapper_manager_update_position.rs RUST
43: }
44:
45: if ctx.accounts.vault.financials.total_aum() > 0 {
46:     match &update {
47:         PositionUpdate::AddTokenAccountBalance { .. } => {}
48:         PositionUpdate::TrackKaminoVaultShares { .. } => {}
49:         PositionUpdate::UpsertKaminoObligationReservePriceMappings { .. } => {}
50:         PositionUpdate::AddKaminoObligationEntry(entry) => {
51:             validate_live_kamino_obligation_has_no_debt(
52:                 &ctx.accounts.vault,
53:                 entry,
54:                 ctx.remaining_accounts,
55:             )?;
56:         }
57:         _ => return Err(ErrorCodeVaults::LivePositionSetupOnlySupportsAdditions.into())
58:     },
59: }
60:
61: action_execution::execute_managed_action(
```

DESCRIPTION

A Kamino lending obligation becomes an AUM-bearing tracked position through **add_kamino_obligation_entry**.

The function confirms the obligation account is owned by the Kamino lending program and by the vault's Squads vault, that the quote price chains to the vault underlying, and that each active reserve has a price mapping in the obligation's lending market. The lending market is a caller-supplied account, fixed when the obligation is created through Kamino **init_obligation** (driven by the manager via the Squads policy and validated by **validate_init_obligation**).

Kamino lending markets are created permissionlessly. The market-initialization handler in the Kamino Lending program makes the caller the market owner:

RUST

```
pub struct InitLendingMarket<'info> {
    #[account(mut)]
    pub lending_market_owner: Signer<'info>,

    #[account(zero)]
    pub lending_market: AccountLoader<'info, LendingMarket>,

    /// CHECK: Seed checked
    #[account(
        seeds = [seeds::LENDING_MARKET_AUTH, lending_market.key().as_ref()],
        bump
    )]
    pub lending_market_authority: AccountInfo<'info>,

    pub system_program: Program<'info, System>,
    pub rent: Sysvar<'info, Rent>,
}
```

The market owner sets the reserve configuration (interest curve, fees, withdrawal caps, reserve status) and can operate borrower obligations against the market's reserves.

Registration is reachable by the manager and the allocator through **wrapper_manager_update_position**, including on a vault that already holds user funds. When **vault.financials.total_aum() > 0**, **AddKaminoObligationEntry** has its own permitted match arm and reaches the update, gated only by **validate_live_kamino_obligation_has_no_debt**:

RUST

```
PositionUpdate::AddKaminoObligationEntry(entry) => {
    validate_live_kamino_obligation_has_no_debt(&ctx.accounts.vault, entry, ctx.remaining_accounts)?;
}
_ => return Err(ErrorCodeVaults::LivePositionSetupOnlySupportsAdditions.into()),
```

validate_live_kamino_obligation_has_no_debt inspects the obligation being registered: it requires that obligation to have no outstanding borrows and to be owned by the Squads vault. It reads only that obligation's own borrows and does not look at the lending market or at any other obligation in it. Governance is not required on this path.

Once the obligation is tracked, the manager moves vault assets into it through the Squads policy with a Kamino deposit; **validate_deposit_liquidity** authorizes the deposit and also performs no lending-market check. The manager can point this vault-owned obligation at a lending market they created and control. The reserve and collateral configuration of that market is fully under the manager, so the deposited liquidity can be drained or made unredeemable by the vault. Registration on a live vault is reached through the manager-direct wrapper rather than a governance proposal, so LP holders have no Reject vote or OptOut withdrawal before the funds are deployed, and no recourse afterward.

RECOMMENDATION

Require governance for adding a Kamino lending obligation to a vault that already holds user funds.

Gate `add_kamino_obligation_entry` so that when the vault has non-zero AUM and non-zero LP supply the registration is permitted only through the governance-proposal path, and not through the manager or allocator direct wrapper. This matches the trust boundary the codebase already applies for live-vault setup additivity and gives LP holders the opportunity to reject or opt out of a deployment into an unvetted market.

Permit the manager-direct path only when the vault has zero AUM and zero LP supply, where no existing holders are exposed and future depositors can inspect the deployed positions before depositing.

FIX REVIEW NOTES

Fixed in PR 3886. Kamino lending-market registration (`AddKaminoObligationEntry`) on a live vault now requires a governance proposal rather than a direct manager action.

M04: Kamino farm reward mints are never checked for priceability, an unpriceable reward freezes recalc for the whole vault



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/kamino/farms_shared.rs:L425-L438 [↗](#)

RELEVANT SNIPPET

```
> _farms_shared.rs RUST
423:     }
424:
425:     let reward_mint = projected_rewards
426:         .projected_reward_infos
427:         .get(reward_index)
428:         .ok_or(ErrorCodeVaults::InvalidAccountData)?
429:         .reward_mint;
430:     let reward_price =
431:         resolve_mint_price(mint_prices, &reward_mint, exponent_prices, clock.current_slot)?;
432:     let reward_value = Number::from_natural_u64(net_reward_amount)
433:         .checked_mul(&reward_price)
434:         .ok_or(ErrorCodeVaults::MathOverflow)?;
435:     total_reward_aum = total_reward_aum
436:         .checked_add(&reward_value)
437:         .ok_or(ErrorCodeVaults::MathOverflow)?;
438: }
439:
440: Ok(total_reward_aum.floor_u64())
```

DESCRIPTION

validate_stake is the only path that registers a non-delegated Kamino farm as a vault strategy position. It performs one priceability check, on the staking mint only:

```
RUST
validate_priceable_mint(vault, &farm_state.underlying_mint)?;
```

The farm's **reward_infos[].reward_mint** entries are never checked. The farm is then tracked.

A reward mint can only be priced if it is already in the vault's tracked-mint set.

`calculate_reward_aum_in_base` prices each reward with a non-zero projected unclaimed amount through `resolve_mint_price`, which looks the mint up in `mint_prices` and errors if absent:

RUST

```
let (_, price_id) = mint_prices
    .iter()
    .find(|(tracked_mint, _)| tracked_mint == mint)
    .ok_or(ErrorCodeVaults::PriceNotFound)?;
```

`mint_prices` is built by `collect_tracked_mint_prices` from exactly the vault's `TokenEntry` mints plus its `TokenAccount` strategy-position mints. There is no separate reward-price registry: a reward mint is priceable only if it coincidentally is also a deposit-asset mint or a token-account-position mint, reusing that mint's `Priceld`.

When a tracked farm has an accrued reward in a mint the vault cannot price, `PriceNotFound` propagates out of `recalculate_aum_in_base`. Every user-facing flow that recalcs (`deposit_liquidity`, `queue_withdrawal`, `execute_withdrawal`, `fill_withdrawal`, `execute_withdrawal_from_reserves`, `wrapper_execute_withdrawal`) reverts. Pending queued withdrawals cannot be executed; deposits and new queues are blocked. The vault is frozen for all users.

An unpriceable reward arises in two ways. First, at the time the farm is staked: its existing reward set may already include a mint the vault has no price for, and nothing checks this. Second, after the farm is tracked: the farm admin can add a new reward token at any time via Kamino's `initialize_reward` (a single farm-admin instruction, up to 10 rewards). Either way the vault freezes the next time that reward accrues.

RECOMMENDATION

Make `calculate_reward_aum_in_base` skip an unpriceable reward (treat its contribution as zero) instead of letting `resolve_mint_price` return `PriceNotFound`. Skip only on the price-not-found case, not on other errors (overflow, stale data), which must still propagate. This removes the freeze in every case: a reward mint that is unpriceable at staking time and one added later via `initialize_reward` are both handled the same way, so no registration-time reward-mint check is needed.

The trade-off: a skipped reward is valued at zero, so AUM is under-counted while the mint stays unpriceable (deposits mint more LP per dollar). When a price is later registered with the position still active, AUM jumps by the now-counted accrued reward. Anyone who deposits during the unpriceable window therefore captures reward value that accrued before they joined, and anyone who withdraws during it forfeits their share.

FIX REVIEW NOTES

Fixed in PR 3888. `calculate_reward_aum_in_base` now swallows only `PriceNotFound`, treating an unpriceable reward as zero; all other errors still propagate, so recalculation fails closed rather than freezing for the whole vault.

M05: Several interaction validators compute `aum_diff` without classifying the source/destination ATA's bucket (reserve vs position), producing values that diverge from the real change to `aum_in_base_in_positions`

**LOCATION**

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validations/token.rs:L11-L44 [🔗](#)

RELEVANT SNIPPET

```
>_ token.rs RUST

 9: pub const TOKEN_2022_PROGRAM_ID: Pubkey = anchor_spl::token_2022::ID;
10:
11: pub fn validate(
12:     vault: &VaultValidationView<'_>,
13:     program_id: &Pubkey,
14:     data: &[u8],
15:     account_indexes: &[u8],
16:     accounts: &[AccountInfo],
17:     tracked: &BTreeSet<Pubkey>,
18: ) -> Result<Option<ValidationResult>> {
19:     if *program_id != TOKEN_PROGRAM_ID && *program_id != TOKEN_2022_PROGRAM_ID {
20:         return Ok(None);
21:     }
22:
23:     // This validator intentionally covers only SPL Transfer / TransferChecked.
24:     // Mint/Burn-driven share flows (such as direct Kamino vault deposits and
25:     // withdraws) need dedicated validation paths that can compare balance
26:     // watermarks instead of trusting a transfer amount field.
27:
28:     // Validate destination is a tracked account.
29:     let destination_index = token_destination_account_index(data, account_indexes)?;
30:     validate_tracked_account_index(destination_index, accounts, tracked)?;
31:
32:     // Resolve transfer direction and the tracked key.
33:     let (tracked_key, is_deposit) =
34:         resolve_transfer_direction(data, account_indexes, accounts, tracked)?;
35:
36:     // Find the price_id for the tracked token account.
37:     let balance = find_token_price_id(vault, &tracked_key)?;
38:
39:     // Parse amount and compute AUM diff.
40:     let token_amount = deserialize_token_amount(data)?;
41:     let aum_diff = calculate_token_aum_diff(vault, token_amount, balance, is_deposit)?
```

>_ token.rs

RUST

```
42: ;
43:     Ok(Some(ValidationResult::new(aum_diff, Vec::new())))
44: }
45:
46: /// Parse the destination account index from a Transfer / TransferChecked instruction.
```

DESCRIPTION

Tracked token accounts fall in one of two buckets. **TokenEntry.token_squads_account** balances are summed into **vault.financials.aum_in_base** (reserves). **TokenAccountEntry.balances[].token_account** balances are summed into **vault.financials.aum_in_base_in_positions** (positions, alongside obligations, farms, CLMM positions, etc.). The reserves invariant enforced inside the hook is **aum_in_base_in_positions + cumulative_aum_diff <= free_aum**, so each validator's **aum_diff** must reflect the real change to **aum_in_base_in_positions** produced by its inner instruction.

The real change is determined by the buckets of both sides of the move:

- Cross-bucket (Reserve to Position, or Position to Reserve): **aum_in_base_in_positions** changes by **⊗amount-in-base**.
- Within-bucket (Position to Position, or Reserve to Reserve): **aum_in_base_in_positions** does not change.

Several validators record **aum_diff** from one side only and never classify the source or destination ATA holding the moved tokens. The recorded value diverges from reality whenever the unclassified side is in the bucket the validator did not assume. Per-site behavior:

kamino_lend.rs obligation validators. Record **⊗amount-in-base** (deposit/withdraw) or **0** (borrow/repay) based purely on the obligation-side movement. The source ATA (deposit/repay) or destination ATA (withdraw/borrow) is never classified. The obligation itself is counted as a position, so:

Operation	ATA bucket	Real delta	Recorded
Deposit (source)	TokenEntry reserve	+X	+X
Deposit (source)	TokenAccount position	0	+X
Withdraw (dest)	TokenEntry reserve	-X	-X
Withdraw (dest)	TokenAccount position	0	-X
Borrow (dest)	TokenEntry reserve	-X	0
Borrow (dest)	TokenAccount position	0	0
Repay (source)	TokenEntry reserve	+X	0
Repay (source)	TokenAccount position	0	0

Lax-direction mismatches (validator records less restrictive than reality): withdraw to `TokenAccount` records **-X** for a real **0** (cumulative drags more negative than reality, lets subsequent ix overshoot the cap); repay from `TokenEntry` records **0** for a real **+X** (cumulative stays flat while reality grows past the cap).

standard_program.rs::validate_mint_sy and validate_redeem_sy. Record $\boxtimes$ **amount-in-base** based purely on the encoded amount and the price chain of the source (`mint_sy`) or source (`redeem_sy`) ATA. The complementary side (SY destination for mint, base destination for redeem) is not classified. When both sides are in the same bucket, the real delta is **0** while the validator records $\boxtimes$ **X**; when they are in opposite buckets, the recorded value matches reality.

token.rs::validate. Records **+amount-in-base** based on the destination side. The destination must be a tracked `TokenAccount` balance (`TokenEntry` destinations are rejected by `find_token_price_id`). The source is never inspected. For an SPL Transfer between two same-mint balances inside one `TokenAccountEntry` (multiple ATAs of the same mint owned by `squads_vault`, both registered in the entry's **balances**), real delta is **0** while the validator records **+X**. This is the within-bucket case; rare in practice but a real configuration. Direction is conservative (over-states), so it tightens the cap check rather than relaxing it; correctness drift, not exploit.

exponent_core.rs wrappers. Several wrapper validators cast a single-sided amount directly into **aum_diff** with `i64::try_from(amount_base)`, without classifying either side. Whenever both sides are in the same bucket, real delta is **0** while the validator records $\boxtimes$ **X**.

Because the cumulative **aum_diff** passed to `verify_aum_change` no longer represents the true post-instruction value of **aum_in_base_in_positions**, every validator listed above can produce an inaccurate **aum_diff** in some configuration.

RECOMMENDATION

utils.rs already exposes bucket-aware helpers used elsewhere in the codebase:

calculate_position_aum_change_for_reserve_account (used by `kamino_farms.rs` and parts of `clmm.rs`) returns $\boxtimes$ **amount-in-base** only when the named side is a reserve and **0** otherwise;

calculate_reserve_to_position_aum_diff_for_known_amount (used by `kamino_vault.rs` and `titan.rs`) classifies both source and destination and returns the correct signed delta.

Replace each hardcoded **aum_diff** with a bucket-aware helper from **utils.rs**:

- **calculate_position_aum_change_for_reserve_account(vault, ata_key, amount, is_increase)**: returns $\boxtimes$ **amount-in-base** if the ATA is a reserve, **0** if it is a position. Use when only one side of the move is an explicit token account (the other side is a non-ATA position like an obligation or a farm).
- **calculate_reserve_to_position_aum_diff_for_known_amount(vault, source, dest, amount, side)**: classifies both sides and returns the correct signed delta. Use when both sides of the move are explicit token accounts.

Per site:

- **kamino_lend.rs** deposit / withdraw / borrow / repay: single-sided helper. Deposit and repay classify the source ATA; withdraw and borrow classify the destination ATA. Also add **validate_tracked_account_index** on the same ATA to reject untracked ATAs.
- **standard_program.rs::validate_mint_sy** and **validate_redeem_sy**: two-sided helper.
- **token.rs::validate**: two-sided helper with `KnownAmountSide::Destination`.
- **exponent_core.rs::validate_wrapper_strip / validate_wrapper_merge / validate_buy_yt / validate_sell_yt**: pick the helper that matches whether both sides of the move are explicit token accounts or only one is.

FIX REVIEW NOTES

Fixed in PR 3894. The bucket-classification concern is moot: the per-instruction **aum_diff** machinery was removed in favor of end-of-transaction final-balance accounting.

M06: Kamino Lending validator does not constrain the user-side liquidity ATA in deposit/withdraw/borrow/repay, letting the manager hide and restore tracked AUM to dilute LP holders



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validations/kamino_lend.rs:L294-L315 [🔗](#)

RELEVANT SNIPPET

```
>_kamino_lend.rs RUST
292: }
293:
294: fn validate_deposit_liquidity<'info>(
295:     vault: &VaultValidationView<'info>,
296:     data: &[u8],
297:     account_indexes: &[u8],
298:     accounts: &'info [AccountInfo<'info>],
299:     tracked: &BTreeSet<Pubkey>,
300: ) -> Result<Option<ValidationResult>> {
301:     let (kamino_entry, _) = resolve_kamino_entry(vault, account_indexes, accounts, tracked)?;
302:     let (reserve_account, reserve_key) = resolve_instruction_reserve(
303:         account_indexes,
304:         accounts,
305:         DEPOSIT_OR_WITHDRAW_RESERVE_ACCOUNT_INDEX,
306:     )?;
307:
308:     let deposit_amount = deserialize_liquidity_amount(data)?;
309:     let deposit_aum = reserve_aum_in_base_stored(
310:         kamino_entry,
311:         vault.exponent_prices,
312:         &reserve_key,
313:         reserve_account,
314:         deposit_amount,
315:     )?;
316:
317:     // Depositing into Kamino increases AUM deployed into positions.
```

DESCRIPTION

Four sibling validators in **kamino_lend.rs** resolve the obligation and the reserve, then record an obligation-side-only **aum_diff** (☒ **amount-in-base** for deposit/withdraw, **0** for borrow/repay). None of them inspects the user-side liquidity ATA:

- **validate_deposit_liquidity**: **user_source_liquidity** (index 9 of **DepositReserveLiquidityAndObligationCollateral**).
- **validate_withdraw_liquidity**: **user_destination_liquidity** (index 9 of **WithdrawObligationCollateralAndRedeemReserveCollateral**).
- **validate_borrow_liquidity** (via **validate_obligation_reserve_ix**): **user_destination_liquidity** (index 8 of **BorrowObligationLiquidity**).
- **validate_repay_liquidity** (via **validate_obligation_reserve_ix**): **user_source_liquidity** (index 6 of **RepayObligationLiquidity**).

Kamino's own constraints on these ATAs only require **token::mint = reserve_liquidity_mint** and, for deposit/withdraw/borrow, **token::authority = squads_vault**. None of the four enforces that the ATA is one of the accounts the vault actually tracks for AUM.

That gap is the bug. **recalculate_aum_in_base** only walks **vault.token_entries** and **vault.strategy_positions**. A freshly-created squads_vault-PDA-owned ATA that the manager never registered with the vault passes Kamino's mint/authority checks but is invisible to recalc. Move funds into it via Withdraw or Borrow, and the obligation's tracked contribution shrinks while the new liquidity goes uncounted: tracked AUM drops by the moved amount. Move funds back out via Deposit or Repay, and tracked AUM returns to its pre-move value.

Impact is LP dilution. Between the hide and restore steps, the manager calls **deposit_liquidity** against the deflated AUM, minting bonus LP at the lower share price (e.g., 2x as many LP per dollar when half of the obligation's value is hidden). After the restore, the manager redeems those bonus LP at the recovered share price; the extra LP value comes out of the existing holders' diluted shares.

RECOMMENDATION

Validate that the user-side liquidity ATA is in the vault's **tracked** set in each of the four validators. Deposit and repay must check the source index; withdraw and borrow must check the destination index.

RUST

```
const DEPOSIT_USER_SOURCE_INDEX: usize = 9;
const WITHDRAW_USER_DESTINATION_INDEX: usize = 9;
const BORROW_USER_DESTINATION_INDEX: usize = 8;
const REPAY_USER_SOURCE_INDEX: usize = 6;

fn validate_deposit_liquidity(...) -> Result<Option<ValidationResult>> {
    // ... existing obligation + reserve checks ...
    let source_idx = resolve_account_index(account_indexes, DEPOSIT_USER_SOURCE_INDEX)?;
    if let Err(err) = validate_tracked_account_index(source_idx, accounts, tracked) {
        return Err(err);
    }
    // ... existing aum_diff computation ...
}
```

Apply the same shape at the matching index in the other three validators. Untracked user-side ATAs are then rejected outright, closing both the hide and restore halves of the cycle.

FIX REVIEW NOTES

Fixed in PR 3890. All four Kamino Lending obligation operations (deposit, withdraw, borrow, repay) now require the user-side liquidity ATA to be a tracked account.

M07: `wrapper_remove_offer` validator skips destination tracking, letting the manager redirect the cancelled offer's PT / YT / SY payload to any token account



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validation
s/orderbook.rs:L483-L510 [🔗](#)

RELEVANT SNIPPET

>_orderbook.rs

RUST

```
481:
482: /// Validate wrapper_remove_offer instruction (cancel an existing offer)
483: fn validate_wrapper_remove_offer(
484:     vault: &VaultValidationView<'>,
485:     data: &[u8],
486:     account_indexes: &[u8],
487:     accounts: &[AccountInfo],
488:     tracked: &BTreeSet<Pubkey>,
489: ) -> Result<Option<ValidationResult>> {
490:     let context = load_wrapper_orderbook_context(vault, account_indexes, accounts, tra
cked)?;
491:
492:     // Parse wrapper instruction args (1-byte discriminator)
493:     let mut args_data = data
494:         .get(1..)
495:         .ok_or(ErrorCodeVaults::UnsupportedInstruction)?;
496:     let args = WrapperRemoveOfferArgs::deserialize(&mut args_data)
497:         .map_err(|_| ErrorCodeVaults::UnsupportedInstruction)?;
498:     validate_wrapper_orderbook_entry(&context)?;
499:
500:     // Cancelling an offer returns funds to the user escrow
501:     // The AUM will be updated on the next recalculate_aum call
502:
503:     Ok(Some(ValidationResult::new(
504:         0,
505:         vec![VaultMutation::UntrackOrderbookOffer {
506:             orderbook: context.orderbook_key,
507:             offer_index: args.offer_idx,
508:         }],
509:     )))
510: }
511:
512: /// Validate wrapper_withdraw_funds instruction (withdraw from user escrow)
```

DESCRIPTION

`validate_wrapper_remove_offer` reports `aum_diff = 0` and emits only an `UntrackOrderbookOffer` mutation, with no check on the trader-side token accounts that receive the cancelled offer's PT / YT / SY payload. `aum_diff = 0` is the correct value when the destination is a tracked `TokenAccount` strategy position (position -> position keeps `aum_in_base_in_positions` invariant, both the offer in `offer_idx_vec` and the receiving `TokenAccount` contribute to that field). The validator never enforces this destination constraint, however, so a manager can route the cancelled offer's payload to any token account they choose, including a manager-owned wallet, while the hook records no AUM change.

The inline comment above `validate_wrapper_remove_offer` gives a different justification for the same `aum_diff = 0` choice: "Cancelling an offer returns funds to the user escrow / The AUM will be updated on the next `recalculate_aum` call." Under that semantics, the offer's value would move into the `OrderbookEntry`'s `user_escrow` leg of `calculate_aum_in_base` and stay inside the position, so destination tracking would also be unnecessary.

The orderbook does not do this on a user-facing cancel. `UserEscrow::update_on_offer_removal` branches on `withdraw_funds: bool`:

RUST

```
if withdraw_funds {
    if offer.get_type() == OfferType::Sellyt && !offer.is_virtual() {
        self.staked_yt_amount = self.staked_yt_amount.checked_sub(offer.amount).unwrap();
    }
    // pt_amount / sy_amount / yt_amount are NOT credited
} else if offer.amount != 0 {
    match offer.get_type() {
        OfferType::BuyYt => {
            if offer.is_virtual() { self.pt_amount = self.pt_amount + offer.amount; }
            else { self.sy_amount = self.sy_amount + offer.amount; }
        }
        OfferType::Sellyt => {
            if offer.is_virtual() { self.sy_amount = self.sy_amount + offer.amount; }
            else { self.yt_amount = self.yt_amount + offer.amount; }
        }
    }
}
```

The credit-back branch (`withdraw_funds = false`) is reached only by `RawOrderbook::post_offer` and `RawOrderbook::market_offer` during mid-fill auto-cleanup of fully-consumed counter offers (both call sites annotated `// store funds in user escrow`). The user-facing `remove_offer` the wrapper invokes (`wrapper_remove_offer` -> `self_cpi::do_cpi_remove_offer` -> single-byte discriminator 4) calls `RawOrderbook::remove_offer` with `withdraw_funds = true` hardcoded at `remove_offer.rs:271`:

RUST

```
let offer =
    orderbook_raw.remove_offer(previous_offer_idx, offer_idx, sy_exchange_rate, true)?;
//
//
//
...
ctx.accounts.do_transfers(ctx.remaining_accounts, &[&signer_seeds], offer, is_expired)?;
```

`do_transfers` then runs an SPL Token Transfer from `token_escrow_xx` to the caller-supplied `token_xx_trader`, with the orderbook PDA as authority. The destination is declared `UncheckedAccount<'info>` with no owner constraint, and SPL Token only enforces `from.owner == authority`, so the destination owner is free. Neither the position -> position invariant (destination must be a tracked `TokenAccount`) nor the comment's user_escrow framing (cancellation must credit `user_escrow`) is enforced anywhere, and the orderbook makes the second impossible by always taking the `withdraw_funds = true` path. The hook's reserves invariant (`vault.verify_aum_change`) consumes `aum_diff = 0` and is trivially satisfied regardless of where the funds actually went.

RECOMMENDATION

Mirror `validate_wrapper_withdraw_funds`: require `token_sy_trader`, `token_yt_trader`, and `token_pt_trader` to be in `tracked`.

RUST

```
for idx in [
    WRAPPER_REMOVE_OFFER_TOKEN_SY_TRADER_ACCOUNT_INDEX,
    WRAPPER_REMOVE_OFFER_TOKEN_YT_TRADER_ACCOUNT_INDEX,
    WRAPPER_REMOVE_OFFER_TOKEN_PT_TRADER_ACCOUNT_INDEX,
] {
    let account_idx = *account_indexes
        .get(idx)
        .ok_or(ErrorCodeVaults::UnsupportedInstruction)? as usize;
    validate_tracked_account_index(account_idx, accounts, tracked)?;
}
```

Reject calls with `redeem_sy_accounts_until > 0` so the wrapper's `cpi_redeem_sy` step does not run. The SY -> BASE redemption can be performed as a separate Squads instruction.

Keep `aum_diff = 0`. With the redeem step disallowed, the three trader-side destinations are SY / PT / YT, which an orderbook-using vault necessarily tracks as `TokenAccount` strategy positions.

Align the inline comment with the actual `withdraw_funds = true` semantics (funds exit to the trader account; the `user_escrow` credit-back branch is reached only by `post_offer` / `market_offer` mid-fill auto-cleanup).

FIX REVIEW NOTES

Fixed. PR 3890 added `validate_wrapper_exit_trader_accounts_tracked` to the `wrapper_remove_offer` validator, but the architectural refactor in PR 3894 initially removed it, re-opening the issue: the cancelled offer's PT, YT, and SY payload is atomically transferred to caller-supplied `trader_*_token` accounts, which the end-of-transaction final-balance accounting does not constrain because the value never lands in a tracked reserve. After this regression was flagged, the team reintroduced `validate_wrapper_exit_trader_accounts_tracked` in a follow-up commit to PR 3894, so all destination trader accounts are tracked again and the payload can no longer be redirected to an untracked account.

M08: Loopscale **CreateLoan** validator does not bind **borrower** to the vault, letting the manager track a loan whose collateral they control outside the vault and freely move the LP share price



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validation
s/loopscale.rs:L111-L117 [↗](#)

RELEVANT SNIPPET

```
>_loopscale.rs RUST
109: match kind {
110:     // --- Loan instructions ---
111:     LoopscaleInstructionKind::CreateLoan => {
112:         let loan_key = resolve_account_key(account_indexes, accounts, LOAN_ACCOUNT_INDE
EX)?;
113:         Ok(Some(ValidationResult::new(
114:             0,
115:             vec![VaultMutation::TrackLoopscaleLoan { loan: loan_key }],
116:         )))
117:     }
118:     LoopscaleInstructionKind::CloseLoan => {
119:         let loan_key = resolve_account_key(account_indexes, accounts, LOAN_ACCOUNT_INDE
EX)?;
```

DESCRIPTION

When the manager routes a Loopscale **create_loan** instruction through a Squads sync, the validator tracks the new loan PDA as a vault **StrategyPosition::LoopscaleLoan** with no checks at all:

```
RUST
LoopscaleInstructionKind::CreateLoan => {
    let loan_key = resolve_account_key(account_indexes, accounts, LOAN_ACCOUNT_INDEX)?;
    Ok(Some(ValidationResult::new(
        0,
        vec![VaultMutation::TrackLoopscaleLoan { loan: loan_key }],
    )))
}
```

In particular, the validator never inspects the **borrower** account at instruction-account index 2. Upstream account layout ([programs/loopscale/src/instructions/loan/create.rs](#)):

RUST

```
pub struct CreateLoan<'info> {
    pub bs_auth: Signer<'info>,           // index 0
    #[account(mut)] pub payer: Signer<'info>, // index 1
    pub borrower: Signer<'info>,           // index 2 ← uncontrolled by Exponent
    #[account(
        init, payer = payer, space = Loan::SIZE,
        seeds = [borrower.key().as_ref(), params.nonce.to_le_bytes().as_slice()],
        bump,
    )]
    pub loan: AccountLoader<'info, Loan>, // index 3
    pub system_program: Program<'info, System>,
}
```

Crucially, the loan PDA seeds are **[borrower.key, nonce]**, and **Loan::create** writes **self.borrower = borrower.key()**. Whoever signs as **borrower** becomes the durably-stamped owner of the loan. Squads' **execute_transaction_sync_v2** propagates every outer-transaction signer to the inner instruction's signer set, so the manager can include their own EOA wallet as an additional outer-tx signer and pass it as the **borrower** account on the inner **create_loan**. The PDA derives from **manager_wallet**, not from **squads_vault**.

After creation, every value-moving Loopscale loan operation enforces **has_one = borrower** plus **pub borrower: Signer<'info>**:

- **deposit_collateral, withdraw_collateral**
- **borrow_principal, repay_principal**
- **close_loan**
- **update_weight_matrix**

If the loan was created with **borrower = manager_wallet**, only **manager_wallet** can operate it. The Exponent vault, even via Squads, can never touch it. The loan nonetheless remains in **vault.strategy_positions**, and the AUM reader has no check that links the tracked entry back to **squads_vault**:

RUST

```
pub fn calculate_loan_aum_in_base(loan_account, ...) -> Result<u64> {
    if is_account_closed(loan_account) { return Ok(0); }
    require_keys_eq!(*loan_account.owner, LOOPSCALE_PROGRAM_ID, ...);
    // reads loan.collateral[], loan.ledgers[]
    // NO check on loan.borrower
    Ok(collateral_value.checked_sub(&debt_value)?.floor_u64())
}
```

The reader prices the loan as **collateral_value - debt_value** regardless of who **loan.borrower** is. With **borrower = manager_wallet**, the manager controls the loan's collateral slots from outside the Exponent program. By depositing collateral from their own wallet they raise the reader's AUM (phantom inflation) without putting anything into the vault, then by withdrawing they reset it.

This gives a malicious manager unilateral control over the vault's visible AUM, which is the denominator in the LP share price. They can move it up or down at will by adjusting the loan's collateral from their EOA.

RECOMMENDATION

Enforce `borrower == vault.squads_vault` inside the `CreateLoan` validator. The borrower account is at instruction-account index 2.

FIX REVIEW NOTES

Fixed in PR 3890. The Loopscale `CreateLoan` validator now binds `borrower` (read from `accounts[2]`) to `vault.squads_vault`, so the manager can no longer track a loan whose collateral they control outside the vault.

M09: Loopscale **CreateStrategy** validator does not bind **params.lender** to the vault, letting the manager track a strategy whose balance they control outside the vault and freely move the LP share price



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validations/loopscale.rs:L229-L244 [↗](#)

RELEVANT SNIPPET

> _loopscale.rs

RUST

```
227: }
228: // --- Strategy instructions ---
229: LoopscaleInstructionKind::CreateStrategy => {
230:     let strategy_key =
231:         resolve_account_key(account_indexes, accounts, STRATEGY_ACCOUNT_INDEX)?;
232:     // Refuse strategies created with an external yield source (e.g. MarginFi). Pinnin
233:     // `external_yield_source_args = None` at creation, plus the `UpdateStrategy` guar
234:     // below and the reader-side guard, guarantees no tracked Exponent-vault strategy
235:     // can route principal off the Loopscale strategy account into a third-party
236:     // protocol whose risk surface we have not audited.
237:     require_create_strategy_no_external_yield(data)?;
238:     Ok(Some(ValidationResult::new(
239:         0,
240:         vec![VaultMutation::TrackLoopscaleStrategy {
241:             strategy: strategy_key,
242:         }],
243:     )))
244: }
245: LoopscaleInstructionKind::CloseStrategy => {
246:     let strategy_key =
```

DESCRIPTION

When the manager routes a Loopscale **create_strategy** instruction through a Squads sync, the validator tracks the new strategy PDA as a vault **StrategyPosition::LoopscaleStrategy** after a single per-instruction guard (no external yield):

RUST

```
LoopscaleInstructionKind::CreateStrategy => {
    let strategy_key = resolve_account_key(account_indexes, accounts, STRATEGY_ACCOUNT_INDEX)?
    ;
    require_create_strategy_no_external_yield(data)?;
    Ok(Some(ValidationResult::new(
        0,
        vec![VaultMutation::TrackLoopscaleStrategy { strategy: strategy_key }],
    )))
}
```

The validator never inspects **CreateStrategyParams.lender**, which sits at the start of the params payload (data offset 8..40). Upstream layout
([programs/loopscale/src/instructions/strategy/create.rs](#)):

RUST

```
pub struct CreateStrategyParams {
    pub lender: Pubkey, // ← uncontrolled by Exponent validator
    pub origination_cap: u64,
    pub liquidity_buffer: u64,
    pub interest_fee: u64,
    pub origination_fee: u64,
    pub principal_fee: u64,
    pub originations_enabled: bool,
    pub external_yield_source_args: Option<ExternalYieldSourceArgs>,
}
```

Strategy::create writes the field through unchanged ([programs/loopscale/src/state/strategy.rs: self.lender = lender](#)). Loopscale enforces nothing about the relationship between **params.lender** and the call's signers at creation time, so the manager can pass any pubkey, including their own EOA wallet.

After creation, every value-moving Loopscale strategy operation enforces **has_one = lender** plus **pub lender: Signer<'info>**:

- **deposit_strategy** ([programs/loopscale/src/instructions/strategy/deposit.rs](#))
- **withdraw_strategy** ([programs/loopscale/src/instructions/strategy/withdraw.rs](#))
- **update_strategy** ([programs/loopscale/src/instructions/strategy/update.rs](#))
- **close_strategy** ([programs/loopscale/src/instructions/strategy/close.rs](#))

If the strategy was created with **lender = manager_wallet**, only **manager_wallet** (the manager's own keypair) can deposit, withdraw, update, or close it. The Exponent vault, even via Squads, can never touch it because Squads makes the **squads_vault** PDA the signer for inner ix's, not **manager_wallet**. Yet the strategy remains in **vault.strategy_positions**, and the AUM reader has no check that links the tracked entry back to **squads_vault**:

RUST

```
pub fn calculate_strategy_aum_in_base(strategy_account, ...) -> Result<u64> {  
    if is_account_closed(strategy_account) { return Ok(0); }  
    require_keys_eq!(*strategy_account.owner, LOOPSCALE_PROGRAM_ID, ...);  
    // reads strategy.principal_mint, token_balance, current_deployed_amount,  
    // outstanding_interest_amount, interest_per_second, last_accrued_timestamp  
    // NO check on strategy.lender  
}
```

This breaks the implicit assumption that every tracked position is backed by **squads_vault**-controlled value. The manager controls the strategy's balance from outside the Exponent program. By depositing into the strategy from their own wallet they raise the reader's AUM (phantom inflation) without putting anything into the vault, then by withdrawing they reset it.

This gives a malicious manager unilateral control over the vault's visible AUM, which is the denominator in the LP share price. They can move it up or down at will by adjusting the strategy's balance from their EOA.

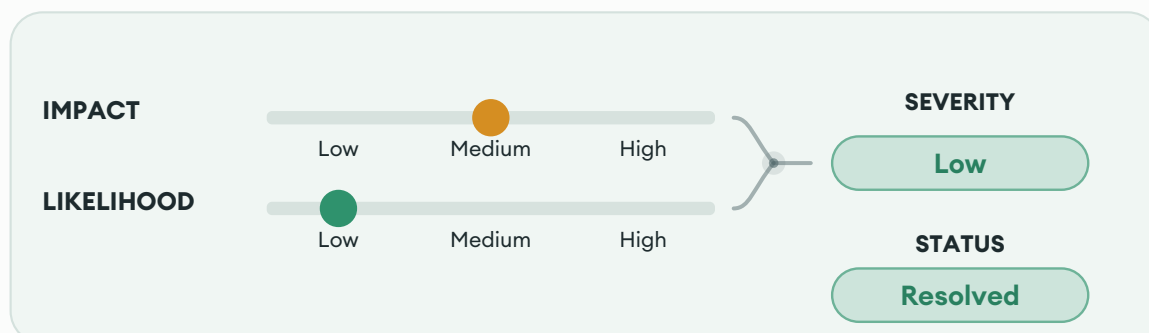
RECOMMENDATION

Enforce **params.lender == vault.squads_vault** inside the **CreateStrategy** validator. The **lender** field sits at instruction data offset 8 to 40.

FIX REVIEW NOTES

Fixed in PR 3890. The Loopscale **CreateStrategy** validator now binds **params.lender** (read from data offset 8) to **vault.squads_vault**, so the manager can no longer track a strategy whose counterparty they control.

L01: `remove_token_entry`'s reference guard only checks orderbook positions, so removing an entry still priced by a non-orderbook position blocks all vault deposits and withdrawals



LOCATION

exponent-monorepo/fb4870a1/solana/programs/exponent_vaults/src/instructions/admin/manage_vault_settings.rs:L448-L481 [↗](#)

RELEVANT SNIPPET

> _manage_vault_settings.rs

RUST

```
446: }
447:
448: fn remove_token_entry(
449:     vault: &mut ExponentStrategyVault,
450:     exponent_prices: &mut ExponentPrices,
451:     mint: Pubkey,
452: ) -> Result<()> {
453:     Self::require_no_pending_withdrawals(vault)?;
454:     // Check that no orderbook position references this token entry
455:     require!(
456:         vault.strategy_positions.iter().all(|pos| {
457:             match pos {
458:                 StrategyPosition::Orderbook(entry) => entry.mint != mint,
459:                 StrategyPosition::TokenAccount(_) => true,
460:                 StrategyPosition::Obligation(_) => true,
461:                 StrategyPosition::YieldPosition(_) => true,
462:                 StrategyPosition::ClmmPosition(_) => true,
463:                 StrategyPosition::LoopscaleLoan(_) => true,
464:                 StrategyPosition::LoopscaleStrategy(_) => true,
465:                 StrategyPosition::KaminoFarm(_) => true,
466:             }
467:         })),
468:         ErrorCodeVaults::TokenEntryInUse
469:     );
470:
471:     let idx = vault
472:         .token_entries
473:         .iter()
474:         .position(|entry| entry.mint == mint)
475:         .ok_or(ErrorCodeVaults::TokenEntryNotFound)?;
476:     let price_id = vault.token_entries[idx].price_id.clone();
477:     price_id.release_price(exponent_prices)?;
478:     vault.token_entries.remove(idx);
479: }
```

>_manage_vault_settings.rs

RUST

```

480:     Ok(())
481: }
482:
483: fn require_no_pending_withdrawals(vault: &ExponentStrategyVault) -> Result<()> {

```

DESCRIPTION

remove_token_entry is meant to refuse removal while a tracked position still uses the mint. The guard only inspects the **Orderbook** variant:

RUST

```

require!(
    vault.strategy_positions.iter().all(|pos| {
        match pos {
            StrategyPosition::Orderbook(entry) => entry.mint != mint,
            StrategyPosition::TokenAccount(_) => true,
            StrategyPosition::Obligation(_) => true,
            StrategyPosition::YieldPosition(_) => true,
            StrategyPosition::ClmmPosition(_) => true,
            StrategyPosition::LoopscaleLoan(_) => true,
            StrategyPosition::LoopscaleStrategy(_) => true,
            StrategyPosition::KaminoFarm(_) => true,
        }
    }),
    ErrorCodeVaults::TokenEntryInUse
);

```

Every non-orderbook arm returns **true**, so the check passes even when an **Obligation**, **YieldPosition**, **ClmmPosition**, **LoopscaleLoan**, **LoopscaleStrategy**, or **KaminoFarm** still prices through **mint**. (**TokenAccount** is safe by construction: **add_token_entry** and **add_token_account_balance** keep token-entry and token-account mints disjoint.)

After the guard passes, **release_price** runs and the entry is removed from **vault.token_entries**; proposal execution does not recalculate AUM, so the removal persists. **collect_tracked_mint_prices** then omits **mint**, and a position reader calling **resolve_mint_price(mint)** returns **PriceNotFound**, reverting **recalculate_aum_in_base** and every deposit and withdrawal path that depends on it.

On a funded vault the removal is performed through a governance proposal. Re-adding the token entry through a later proposal restores the price map and AUM recalculation. The disruption is temporary. **remove_token_account_entry** and **remove_token_account_balance** perform no reference check and produce the same failure.

RECOMMENDATION

Cover every variant that can price through a mint:

RUST

```
require!(
  vault.strategy_positions.iter().all(|pos| match pos {
    StrategyPosition::Orderbook(e) => e.mint != mint,
    StrategyPosition::TokenAccount(_) => true, // disjoint by construction
    StrategyPosition::Obligation(o) => !o.references_mint(&mint),
    StrategyPosition::YieldPosition(e) => e.underlying_mint != mint && e.terminal_mint !=
mint,
    StrategyPosition::ClmmPosition(e) => e.token_a_mint != mint && e.token_b_mint != mint,
    StrategyPosition::LoopscaleLoan(e) => !loan_references_mint(e, &mint),
    StrategyPosition::LoopscaleStrategy(e) => !strategy_references_mint(e, &mint),
    StrategyPosition::KaminoFarm(e) => !farm_references_mint(e, &mint),
  }),
  ErrorCodeVaults::TokenEntryInUse
);
```

FIX REVIEW NOTES

Fixed in PR 3890. The **remove_token_entry** guard now blocks removal whenever any tracked Loopscale loan or strategy or Kamino obligation or farm exists. Dynamic-mint positions always block; static-mint positions that do not route through **resolve_mint_price** never block.

L02: Kamino-vault share price has no staleness bound, so deposits and withdrawals can be priced against a lagged AUM



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/price_sources/kamino_vault.rs:L35-L81 [🔗](#)

RELEVANT SNIPPET

```
> _kamino_vault.rs RUST

33:
34: #[inline(never)]
35: pub fn read_price<'info>(  
36:     last_updated_slot: u64,  
37:     interface_accounts: &[Pubkey],  
38:     accounts: &'info [AccountInfo<'info>],  
39:     clock: &Clock,  
40: ) -> Result<Option<Number>> {  
41:     if last_updated_slot == clock.slot {  
42:         return Ok(None);  
43:     }  
44:   
45:     require!(  
46:         interface_accounts.len() == 1,  
47:         ErrorCodeVaults::MissingPriceAccount  
48:     );  
49:     let expected_vault = &interface_accounts[0];  
50:   
51:     let vault_account_info = accounts  
52:         .iter()  
53:         .find(|acc| acc.key == expected_vault)  
54:         .ok_or(ErrorCodeVaults::MissingPriceAccount)?;  
55:     require_keys_eq!(  
56:         *vault_account_info.owner,  
57:         kamino_vault_cpi::ID,  
58:         ErrorCodeVaults::InvalidAccountData  
59:     );  
60:   
61:     let vault_data = vault_account_info  
62:         .try_borrow_data()  
63:         .map_err(|_| ErrorCodeVaults::InvalidAccountData)?;  
64:     let required_len = 8usize  
65:         .checked_add(size_of::<VaultState>())  
66:         .ok_or(ErrorCodeVaults::MathOverflow)?;  
67:     require!(  
68:         vault_data.len() >= required_len,
```


>_kamino_vault.rs

RUST

```

69:         ErrorCodeVaults::InvalidAccountData
70:     );
71:     require!(
72:         vault_data[..8] == VaultState::DISCRIMINATOR[..],
73:         ErrorCodeVaults::InvalidAccountData
74:     );
75:
76:     let vault: &VaultState = bytemuck::from_bytes(&vault_data[8..required_len]);
77:
78:     let exchange_rate = compute_share_price(vault.prev_aum_sf, vault.shares_issued)?;
79:
80:     Ok(Some(exchange_rate))
81: }
82:
83: /// Pure share-price computation.

```

DESCRIPTION

kamino_vault::read_price computes the Kamino-vault share price as **prev_aum_sf / shares_issued**, read directly from the Kamino **VaultState** account.

RUST

```

let vault: &VaultState = bytemuck::from_bytes(&vault_data[8..required_len]);
let exchange_rate = compute_share_price(vault.prev_aum_sf, vault.shares_issued)?;

Ok(Some(exchange_rate))
}

/// Pure share-price computation.
/// Price = prev_aum_sf / shares_issued. `prev_aum_sf` is already net of
/// `pending_fees_sf` per Kamino's bookkeeping invariant.
pub fn compute_share_price(prev_aum_sf: u128, shares_issued: u64) -> Result<Number> {
    let prev_aum = Fraction::from_bits(prev_aum_sf);
    let prev_aum_num = precise_number_from_fraction(prev_aum);

    let shares = Number::from_natural_u64(shares_issued);
    prev_aum_num
        .checked_div(&shares)
        .ok_or_else(|| error!(ErrorCodeVaults::MathOverflow))
}

```

In Kamino's kvault program, **prev_aum_sf** is written only by the **update_prev_aum** sites (**deposit**, **withdraw**, **redeem_in_kind**, **charge_fees**, **give_up_pending_fee**) in **programs/kvault/src/operations/vault_operations.rs**. Each is preceded by **charge_fees**, which recomputes **compute_aum(invested) = token_available + invested - pending_fees** (**programs/kvault/src/state.rs**). Between those mutations **prev_aum_sf** is frozen while the Kamino vault's lending positions keep accruing interest. So **prev_aum_sf** lags the true live AUM by the interest accrued since the last Kamino mutation.

read_price does not bound this lag. Its only guard, **if last_updated_slot == clock.slot { return Ok(None); }**, deduplicates Exponent-cache writes within a slot and says nothing about Kamino-state

age. **update_price** copies the current **prev_aum_sf** into the Exponent cache but does not advance it, so the staleness window equals the time between external Kamino cranks and is unbounded in code.

A stale-low **prev_aum_sf** over-mints LP on deposit and under-pays on withdrawal. The per-event value is bounded by the interest accrued since the last Kamino crank, so it is negligible for an actively cranked vault and grows unbounded for an inactive one.

RECOMMENDATION

Add a Kamino-side freshness gate in **read_price**. The **VaultState** already deserialized in **read_price** exposes **last_fee_charge_timestamp: u64**, the unix-seconds watermark of the last **charge_fees**, which Kamino updates on every **deposit**, **withdraw**, **invest**, **redeem**, and explicit fee charge. Reject the price when that watermark is older than a bounded window relative to the Solana clock:

RUST

```
const KAMINO_VAULT_MAX_STALENESS_SECONDS: i64 = 600; // tune per deployment

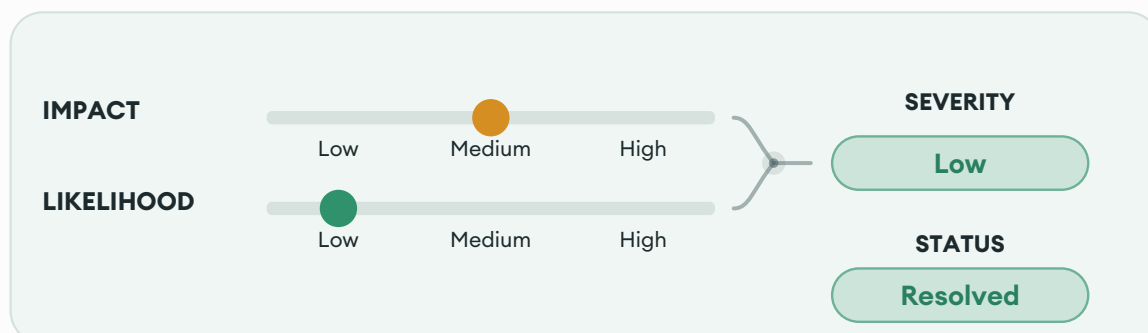
let vault: &VaultState = bytemuck::from_bytes(&vault_data[8..required_len]);
let age = clock
    .unix_timestamp
    .saturating_sub(vault.last_fee_charge_timestamp as i64);
if age > KAMINO_VAULT_MAX_STALENESS_SECONDS {
    return Err(ErrorCodeVaults::StalePriceData.into());
}
let exchange_rate = compute_share_price(vault.prev_aum_sf, vault.shares_issued)?;
```

With this gate, a deposit or withdrawal can only be priced after the Kamino vault has been cranked recently. The crank advances **prev_aum_sf** to the post-accrual value and realizes the accrued interest for all existing LP holders proportionally, so the share math runs against a current AUM and the per-event mispricing is bounded by the configured window instead of by external crank cadence.

DEVELOPER RESPONSE

Acknowledged. The team is waiting on Kamino to expose a refresh primitive. The per-transaction loss potential is small, and an interim on-chain mitigation would require a layout change.

L03: Kamino Lending collateral-withdraw validator trusts caller-declared **collateral_amount**, which Kamino silently caps to deposited collateral, letting the manager bypass the reserves invariant



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validations/kamino_lend.rs:L322-L348 [↗](#)

RELEVANT SNIPPET

```

>_ kamino_lend.rs RUST
320: }
321:
322: fn validate_withdraw_liquidity<'info>(
323:     vault: &VaultValidationView<'>,
324:     data: &[u8],
325:     account_indexes: &[u8],
326:     accounts: &'info [AccountInfo<'info>],
327:     tracked: &BTreeSet<Pubkey>,
328: ) -> Result<Option<ValidationResult>> {
329:     let (kamino_entry, _) = resolve_kamino_entry(vault, account_indexes, accounts, tracked)?;
330:     let (reserve_account, reserve_key) = resolve_instruction_reserve(
331:         account_indexes,
332:         accounts,
333:         DEPOSIT_OR_WITHDRAW_RESERVE_ACCOUNT_INDEX,
334:     )?;
335:
336:     let collateral_amount = deserialize_liquidity_amount(data)?;
337:     let liquidity_amount = reserve_collateral_to_liquidity(reserve_account, collateral_amount)?;
338:     let withdraw_aum = reserve_aum_in_base_stored(
339:         kamino_entry,
340:         vault.exponent_prices,
341:         &reserve_key,
342:         reserve_account,
343:         liquidity_amount,
344:     )?;
345:
346:     let aum_diff = -i64::try_from(withdraw_aum).map_err(|_| ErrorCodeVaults::MathOverflow)?;
347:     Ok(Some(ValidationResult::new(aum_diff, Vec::new())))
348: }
349:
350: fn validate_borrow_liquidity<'info>(

```

DESCRIPTION

validate_withdraw_liquidity is the post-CPI validator for the Kamino Lending instruction **withdraw_obligation_collateral_and_redeem_reserve_collateral** (both the v1 and v2 discriminators). It reads the instruction's **collateral_amount** argument from the inner-instruction data, converts it through the reserve collateral exchange rate, prices it, and returns the result as a negative position-side delta:

RUST

```
let collateral_amount = deserialize_liquidity_amount(data)?;
let liquidity_amount = reserve_collateral_to_liquidity(reserve_account, collateral_amount)?;
let withdraw_aum = reserve_aum_in_base_stored(
    kamino_entry, vault.exponent_prices, &reserve_key, reserve_account, liquidity_amount,
)?;
let aum_diff = -i64::try_from(withdraw_aum).map_err(|_| ErrorCodeVaults::MathOverflow)?;
Ok(Some(ValidationResult::new(aum_diff, Vec::new())))
```

The declared **collateral_amount** is never compared against the obligation's actual collateral position. Kamino Lending, however, silently caps it. In the Kamino Lending program ([programs/klend/src/lending_market/lending_operations.rs](#)), function **withdraw_obligation_collateral**, the borrows-empty branch is:

RUST

```
let withdraw_amount = if is_borrows_empty {
    if collateral_amount == u64::MAX {
        collateral.deposited_amount
    } else {
        collateral.deposited_amount.min(collateral_amount) // silent cap, no error
    }
} else { /* borrows-present path: errors with WithdrawTooLarge on over-large amount */ };
```

For an obligation with no active borrows (the normal shape of a vault that simply lends an asset into Kamino for yield), Kamino withdraws **min(collateral_amount, deposited_amount)** and does not error when the declared value is larger. The validator instead values the full caller-declared **collateral_amount**. With that argument set far above the obligation's **deposited_amount** (any finite value whose priced result stays at or below **i64::MAX**), the reported negative **aum_diff** is far larger in magnitude than the value that actually left the position. The validator parses the obligation in sibling paths (the refresh and borrow/repay validators read it through **obligation_active_reserves**), so the obligation's **deposited_amount** for the withdraw reserve is available at validation time but is not consulted to bound the magnitude.

The hook re-checks the reserves cap after each inner instruction, and that check is single-sided. **verify_aum_change** computes:

RUST

```
let new_in_positions = current_in_positions.checked_add(i128::from(aum_diff))?;
require!(new_in_positions <= effective_free_aum, ErrorCodeVaults::ReservesExceeded);
```

There is no lower bound on **new_in_positions**. The validate-interaction hook accumulates each inner instruction's **aum_diff** into **cumulative_aum_diff** and re-evaluates **verify_aum_change** after each

inner instruction. A single over-stated withdraw drags **cumulative_aum_diff** arbitrarily negative; each subsequent inner instruction in the same Squads sync transaction that deploys reserves into another tracked position contributes a positive **aum_diff** that only brings the running total back toward zero, so every check still passes while real positions are pushed past the **reserves_share_bp** ceiling.

RECOMMENDATION

A complete fix requires the validator to know the pre-interaction collateral. That value is not on-chain post-CPI, so it would have to be persisted in vault state (a per-reserve **deposited_amount** on **KaminoObligationEntry**) and advanced on every path that changes the obligation, both the hook and **recalculate_aum_in_base**, with attention to staleness and to double-counting across multiple interactions in one transaction. That is the correct fix but a substantial one.

A smaller mitigation bounds the error instead of eliminating it, and applies to any Kamino integration that silently caps a declared amount to the available balance. The over-statement is **(declared - actual) * price**, and the cap only fires on a full drain, where **declared >= available** and the position ends at zero. The validator already determines a full drain from post-CPI state, since it parses the obligation through **obligation_active_reserves** in sibling paths: a full drain leaves the withdraw reserve absent from the obligation's active deposits. Restrict a full drain to a declared amount no greater than a small constant **K** (for example one collateral unit):

RUST

```
let declared = deserialize_liquidity_amount(data)?;  
let fully_drained = !obligation_active_reserves(obligation_account)?.0.contains(&reserve_key);  
if fully_drained {  
    require!(declared <= MAX_FULL_DRAIN_DECLARED, ErrorCodeVaults::UnsupportedInstruction);  
}
```

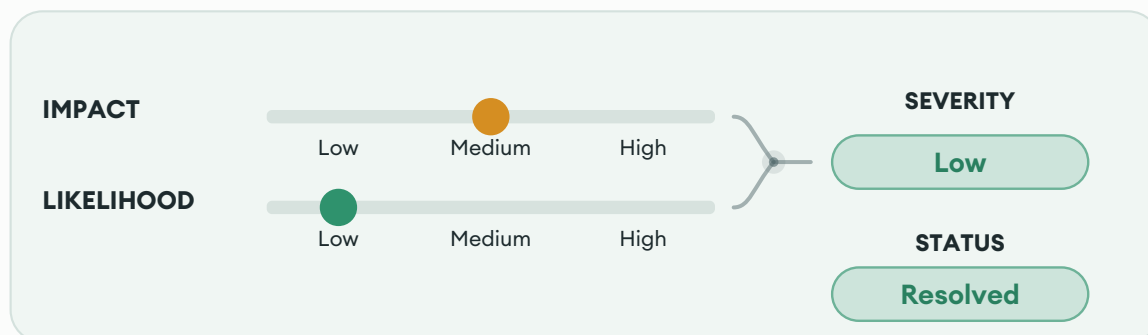
A full drain implies **available <= declared**, so with **declared <= K** the actual amount withdrawn is also at most **K** and the over-statement is at most **K * price**: one collateral unit of value per instruction, economically negligible and unusable for a reserves-cap bypass. Partial withdrawals are untouched and remain exact, because the cap cannot fire while the position retains a balance.

The cost is a UX change for a legitimate full exit of a position holding more than **K** units. The manager issues two inner instructions in the Squads transaction: a partial withdrawal that takes all but a residual of at most **K**, then a second instruction that drains the residual under the **declared <= K** rule. Single partial withdrawals are unchanged.

FIX REVIEW NOTES

Fixed in PR 3894. The **collateral_amount** silent-cap class is superseded by end-of-transaction final-balance accounting; the validator no longer values a caller-declared amount that Kamino caps.

L04: A positions snapshot that is stale across transactions will let a manager deploy past the reserves cap



LOCATION

[exponent-monorepo/fb4870a1/solana/programs/exponent_vaults/src/state/vault.rs:L1200-L1216](https://github.com/exponent-monorepo/fb4870a1/solana/programs/exponent_vaults/src/state/vault.rs#L1200-L1216)

RELEVANT SNIPPET

```
>_ vault.rs RUST

1198: }
1199:
1200: /// Verify that an AUM change (positive = deploying more, negative = withdrawing)
1201: /// does not cause the deployed positions AUM to exceed the free AUM limit.
1202: pub fn verify_aum_change(&self, aum_diff: i64, aum_locked_for_withdrawals: u64) -> Result<> {
1203:     let effective_free_aum =
1204:         i128::from(self.get_free_aum()?.saturating_sub(aum_locked_for_withdrawals));
1205:
1206:     let current_in_positions = i128::from(self.financials.aum_in_base_in_positions);
1207:     let new_in_positions = current_in_positions
1208:         .checked_add(i128::from(aum_diff))
1209:         .ok_or(ErrorCodeVaults::MathOverflow)?;
1210:
1211:     require!(
1212:         new_in_positions <= effective_free_aum,
1213:         ErrorCodeVaults::ReservesExceeded
1214:     );
1215:     Ok(())
1216: }
1217:
1218: /// Total fee basis points for fast withdrawal (base withdrawal fee + fast withdrawal
    cut).
```

DESCRIPTION

The reserves invariant keeps deployed positions within a fraction of total AUM so a buffer of withdrawable reserves always remains. It is enforced in exactly one function, **verify_aum_change**:

RUST

```
pub fn verify_aum_change(&self, aum_diff: i64, aum_locked_for_withdrawals: u64) -> Result<()>
{
    let effective_free_aum =
        i128::from(self.get_free_aum()?.saturating_sub(aum_locked_for_withdrawals));
    let current_in_positions = i128::from(self.financials.aum_in_base_in_positions);
    let new_in_positions = current_in_positions
        .checked_add(i128::from(aum_diff))
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    require!(
        new_in_positions <= effective_free_aum,
        ErrorCodeVaults::ReservesExceeded
    );
    Ok(())
}
```

The baseline **current_in_positions** reads **self.financials.aum_in_base_in_positions**. The interaction hook calls **verify_aum_change** once per inner instruction, with a cumulative that it initializes to zero on every invocation, and it never writes **aum_in_base_in_positions** back:

RUST

```
let mut cumulative_aum_diff = 0i64;

for ix in &inner_instructions.instructions {
    let result = hook_validation::validate_instruction(...)?;
    cumulative_aum_diff = cumulative_aum_diff
        .checked_add(result.aum_diff)
        .ok_or(ErrorCodeVaults::MathOverflow)?;
    vault.verify_aum_change(cumulative_aum_diff, aum_locked_for_withdrawals)?;
    hook_validation::apply_vault_mutations(vault, &mut validation_view, &result.mutations)?;
}
```

aum_in_base_in_positions is written in only one place, the tail of **recalculate_aum_in_base**, which runs from user-facing flows (deposit, withdrawal, fast-withdrawal, fill, the manager wrapper instructions) and not from the hook. The hook omits the recalculation so a Squads transaction does not have to carry every strategy position's accounts, and uses the per-instruction **aum_diff** as the lightweight substitute.

Because the hook resets the cumulative per call and never advances the baseline, two Squads syncs with no recalc-triggering instruction between them both read the same **aum_in_base_in_positions**. The first sync's real position growth is invisible to the second sync's check, so each sync independently receives the full **effective_free_aum - baseline** of deployment headroom. Chained, a manager deploys the entire reserve buffer into positions. The next user-facing flow calls **recalculate_aum_in_base**, which reads the over-deployed reality, stores it, and returns without reverting.

RECOMMENDATION

The bound must persist across transactions instead of resetting each sync. Two implementations work, with different costs:

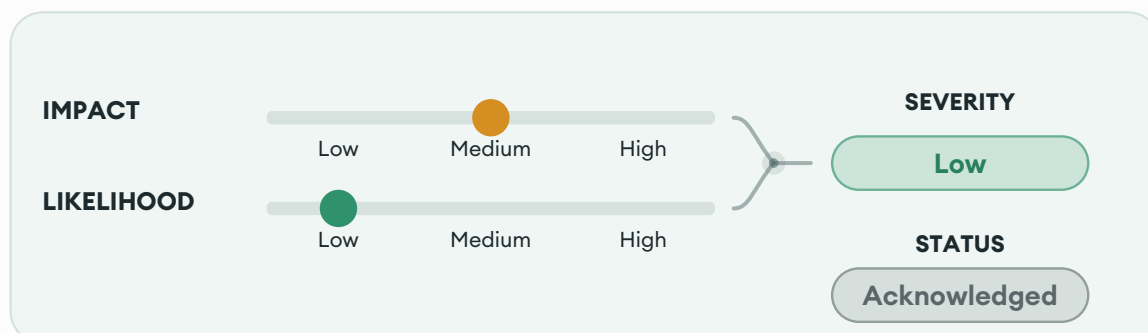
- Recompute the position total inside the hook before checking it. This is exact but requires every strategy position's accounts in the Squads transaction.
- Persist deployment since the last recalc in vault state, add it to the hook's baseline, and clear it when **recalculate_aum_in_base** next runs. This keeps the Squads transaction small and adds one stored field.

The check must stay on the deployment path. **recalculate_aum_in_base** runs on the withdrawal flows and recomputes positions from live state, so gating it would block withdrawals and would trigger on ordinary value changes like collateral price moves or interest accrual.

FIX REVIEW NOTES

Fixed in PR 3880. Cross-transaction staleness is closed by a **FLAG_AUM_REFRESH_REQUIRED** flag set at the end of any state-changing manager sync and asserted at the entry of every other manager-side mutator, cleared only by a new permissionless **refresh_aum** instruction, so two consecutive manager syncs cannot occur without a fresh recalculation between them. A NAV circuit breaker was added as defense-in-depth: it trips when NAV-per-LP drifts beyond a threshold over a window, blocks economic paths, and requires a sentinel to clear. PR 3900 made the breaker threshold per-vault configurable, with a default of 50 bps. The upward-donation grief on the breaker is acknowledged, and the breaker is kept direction-symmetric.

L05: Borrow validators let borrowed funds land in the same account that holds the vault's deposit-funded reserves, making the two indistinguishable for fast-withdrawal and cap accounting



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validations/kamino_lend.rs:L350-L368 [↗](#)

RELEVANT SNIPPET

```
> _kamino_lend.rs RUST
348: }
349:
350: fn validate_borrow_liquidity<'info>(
351:     vault: &VaultValidationView<'>,
352:     data: &[u8],
353:     account_indexes: &[u8],
354:     accounts: &'info [AccountInfo<'info>],
355:     tracked: &BTreeSet<Pubkey>,
356: ) -> Result<Option<ValidationResult>> {
357:     validate_obligation_reserve_ix(
358:         vault,
359:         data,
360:         account_indexes,
361:         accounts,
362:         tracked,
363:         DEPOSIT_OR_WITHDRAW_RESERVE_ACCOUNT_INDEX,
364:         BORROW_V1_FIXED_ACCOUNTS_LEN,
365:         BORROW_V2_FIXED_ACCOUNTS_LEN,
366:         &BORROW_OBLIGATION_LIQUIDITY_V2_DISC,
367:     )
368: }
369:
370: fn validate_repay_liquidity<'info>(
```

DESCRIPTION

Both borrow validators accept any squads_vault-owned ATA as the destination. When the destination is a **TokenEntry.token_squads_account** (the same account that holds the vault's deposit-funded reserves), the borrowed liquidity and the deposit-funded reserves end up commingled in one on-chain account. After the borrow, nothing on-chain distinguishes the two.

That commingling has two consequences for the protocol's accounting model.

Fast-withdrawal. `execute_withdrawal_from_reserves` is open to any LP holder and sources from `TokenEntry.token_squads_account` balances. Any LP holder can therefore redeem against the debt-funded liquidity at the prevailing share price. They get a fair pro-rata payout, but the liquidity they pull is debt-backed; the remaining holders are left holding a more leveraged obligation.

Reserves cap. The obligation's contribution to `aum_in_base_in_positions` is `collateral - debt`. The borrow shrinks the contribution by `X`, so the next recalc writes a lower `aum_in_base_in_positions` and the cap (`aum_in_base_in_positions <= free_aum`) reads `X` more headroom. A follow-up transaction can deploy `X` of additional capital into other positions; gross exposure grows but the cap formula is satisfied.

The protocol's accounting separates vault holdings into reserves (balances in `TokenEntry.token_squads_account`, available for LP withdrawal) and positions (StrategyPosition AUM, including `TokenAccount` strategy positions whose balances are not LP-withdrawable). Borrowed liquidity belongs in the position side: it is offset by a tracked debt liability and should not count toward LP redemption capacity at face value. The validators let the borrow drop debt-funded liquidity into the reserve account instead, commingling it with deposit-funded liquidity for both `execute_withdrawal_from_reserves` and the cap formula.

PROOF OF CONCEPT

Start state: USDC vault with `$500` reserves (in the USDC `TokenEntry`'s `token_squads_account`, call it `X`) and a Kamino obligation with `$500` USDC collateral and `$0` debt. Total AUM `$1,000`.

Manager borrows `$200` USDC via `BorrowObligationLiquidity` with `user_destination_liquidity = X`. Post-borrow: `X` holds `$700`, debt `$200`, obligation contribution `$500 - $200 = $300`. Total still `$1,000`.

- **Fast-withdraw.** Any LP holder calls `execute_withdrawal_from_reserves`, which sources from `X`. The borrowed `$200` is indistinguishable from the original reserves; whichever LP withdraws first walks out with debt-backed liquidity, leaving the leveraged obligation for the holders who did not.

- **Cap relaxation.** The next recalc writes `aum_in_base_in_positions = $300` (down from `$500`). The reserves cap now permits `$200` more deployment than before; a follow-up Squads sync can stake that `$200` into a farm. Gross exposure (collateral + farm = `$500 + $200`) grew, but the cap formula reads `($500 - $200) + $200 = $500` and is satisfied.

`loopscale.rs::BorrowPrincipal` produces the same shape with the Loopscale loan in place of the Kamino obligation.

RECOMMENDATION

Reject borrows whose destination is a `TokenEntry.token_squads_account`. The borrowed liquidity must land in a tracked `TokenAccount` strategy position so it stays separated from deposit-funded reserves.

This forbids borrowing a mint that the vault already holds as a `TokenEntry` (the mint-shadow rule prevents a matching `TokenAccount` of the same mint). If the protocol wants to support same-mint borrows (e.g., a USDC vault borrowing more USDC), the accounting model needs to recognize debt-backed liquidity as a distinct category that is excluded from `execute_withdrawal_from_reserves` sourcing and offsets the obligation's debt in the cap formula. That is a larger change than a validator-level constraint.

DEVELOPER RESPONSE

Acknowledged. The audit re-raised the specific trust-boundary concern: positions are manager-handled (deployment, leverage), while reserves are the liquid bucket that LPs can self-serve through **execute_withdrawal_from_reserves**. During a manager borrow window, the borrowed funds that enter the reserve account are therefore withdrawable by any LP, leaving residual holders backing the leveraged obligation. The team confirmed that this is intended behavior: withdrawals being filled by debt is an accepted part of the design. The risk is acknowledged on this basis.

L06: `update_price` on a TWAP entry delays the next `update_twap_price`, slowing TWAP observations below the configured cadence



LOCATION

[exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/prices.rs:L717-L719](#)

RELEVANT SNIPPET

```
> _prices.rs RUST  
715:     store_price_interface_accounts(interface_accounts_info, &price_interface_accounts)  
716:     ?;  
717: }  
717: if let Some(price) = price {  
718:     apply_price_update(price_entry, price, &clock);  
719: }  
720:  
721: // Move the index forward
```

DESCRIPTION

A TWAP entry has two timestamps. `last_updated_slot` is the same-slot freshness marker AUM math reads. `last_updated_at` is the rate-limit timestamp the TWAP reads to decide whether enough time has passed to accept a new observation.

These should be controlled by different writers. `update_twap_price` is the only call that should ever push `last_updated_at` forward, because it is the only call that adds an observation. `update_price` exists to keep the cache marked fresh between observations so AUM math passes; it should touch `last_updated_slot` only.

Both calls write both fields. The shared helper at the end of `update_prices_inner` is:

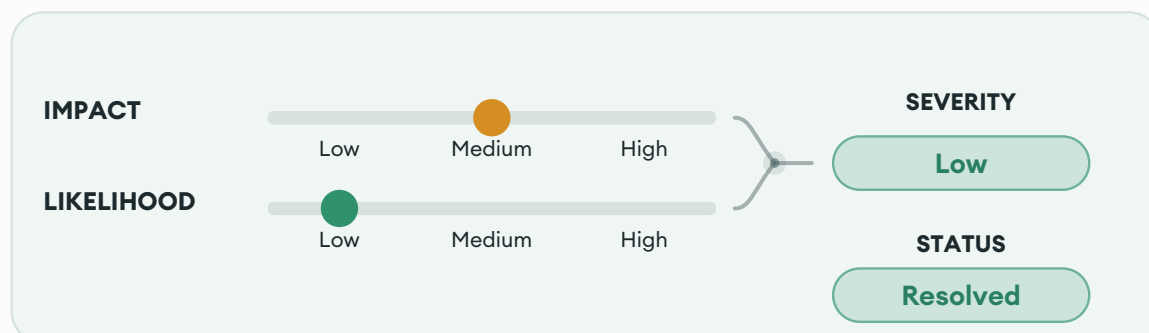
```
RUST  
fn apply_price_update(price_entry: &mut ExponentPrice, price: Number, clock: &Clock) {  
    price_entry.price = price;  
    price_entry.last_updated_slot = clock.slot;  
    price_entry.last_updated_at = clock.unix_timestamp as u64;  
}
```

So if any `update_price` lands between two `update_twap_price` calls, the next `update_twap_price` is delayed. The size of the delay is the offset between the last observation and the `update_price` call.

FIX REVIEW NOTES

Fixed in PR 3899. **update_price** no longer bumps **last_updated_at** on TWAP entries, so the observation cadence remains controlled solely by **update_twap_price**.

L07: Kamino Farms **stake** validator reads the source token balance after the inner CPI has already drained it, producing a wrong **aum_diff** for the **u64::MAX** "stake everything" path



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validation
s/kamino_farms.rs:L113-L159 [↗](#)

RELEVANT SNIPPET

```

>_kamino_farms.rs
111: }
112:
113: fn validate_stake<'info>(
114:     vault: &VaultValidationView<'>,
115:     data: &[u8],
116:     account_indexes: &[u8],
117:     accounts: &'info [AccountInfo<'info>],
118:     tracked: &BTreeSet<Pubkey>,
119: ) -> Result<Option<ValidationResult>> {
120:     let farm_state_account = resolve_account_info(account_indexes, accounts, FARM_STATE_INDEX)?;
121:     let farm_state = read_farm_state(farm_state_account)?;
122:     require!(
123:         !farm_state.is_delegated(),
124:         ErrorCodeVaults::UnsupportedInstruction
125:     );
126:     validate_priceable_mint(vault, &farm_state.underlying_mint)?;
127:
128:     let source_account_index =
129:         resolve_account_index(account_indexes, STAKE_SOURCE_TOKEN_ACCOUNT_INDEX)?;
130:     validate_tracked_account_index(source_account_index, accounts, tracked)?;
131:
132:     let source_account = accounts
133:         .get(source_account_index)
134:         .ok_or(ErrorCodeVaults::UnsupportedInstruction)?;
135:     let encoded_amount = deserialize_u64_at(data, DISCRIMINATOR_LEN)?;
136:     let amount = if encoded_amount == u64::MAX {
137:         parse_token_account_amount(source_account)?
138:     } else {
139:         encoded_amount
140:     };
141:
142:     let user_state = resolve_account_key(account_indexes, accounts, USER_STATE_INDEX)?;
143: }

```

>_kamino_farms.rs

RUST

```

143:     let farm_state_key = *farm_state_account.key;
144:     let aum_diff = calculate_position_aum_change_for_reserve_account(
145:         vault,
146:         source_account.key,
147:         amount,
148:         true,
149:     )?;
150:     let mut mutations = Vec::new();
151:     if !is_kamino_farm_tracked(vault, &farm_state_key, &user_state) {
152:         mutations.push(VaultMutation::TrackKaminoFarm {
153:             farm_state: farm_state_key,
154:             user_state,
155:         });
156:     }
157:
158:     Ok(Some(ValidationResult::new(aum_diff, mutations)))
159: }
160:
161: fn validate_unstake<'info>(<

```

DESCRIPTION

The **validate_interaction** hook runs AFTER the inner instructions have executed, so account reads inside see post-CPI state. **validate_stake** ignores that:

RUST

```

let encoded_amount = deserialize_u64_at(data, DISCRIMINATOR_LEN)?;
let amount = if encoded_amount == u64::MAX {
    parse_token_account_amount(source_account)?
} else {
    encoded_amount
};
let aum_diff = calculate_position_aum_change_for_reserve_account(
    vault, source_account.key, amount, true,
)?;

```

parse_token_account_amount reads the source ATA's current balance. Kamino's **stake** handler resolves the same **u64::MAX** sentinel against the pre-CPI **user_ata.amount** and then SPL-transfers that full amount out of the source. By the time the hook runs, the source balance is **0**, so the validator computes **amount = 0** and **aum_diff = 0** for a stake that actually moved the full reserve balance into the farm.

The hook's reserves-invariant check (**validate_interaction.rs:198-212**) consumes that **aum_diff**, so it sees no change and passes trivially while **aum_in_base_in_positions** has in reality grown by **N * price_in_base**. The manager can drain every tracked reserve ATA into a Kamino farm in one Squads sync without tripping the reserves cap.

RECOMMENDATION

The validator's **u64::MAX** branch might not be fixable without a pre-CPI snapshot, which the hook does not have. The clean fix is to reject the **u64::MAX** in the validator and force the caller to encode the actual amount:

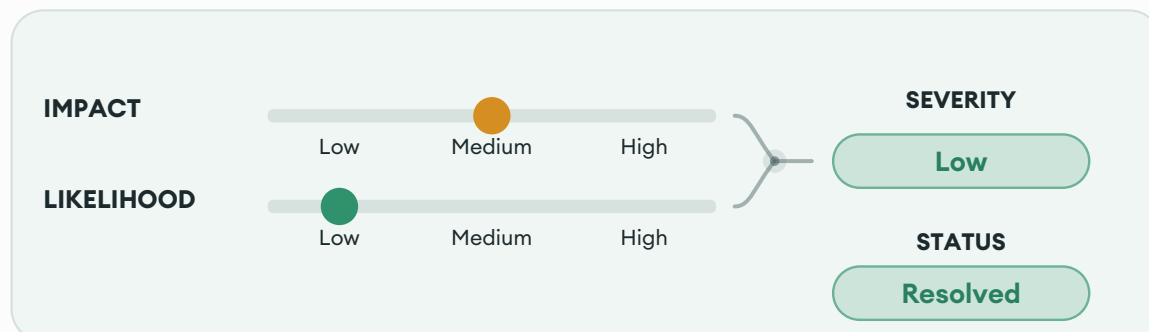
RUST

```
let encoded_amount = deserialize_u64_at(data, DISCRIMINATOR_LEN)?;  
require!(  
    encoded_amount != u64::MAX,  
    ErrorCodeVaults::UnsupportedInstruction  
);  
let amount = encoded_amount;
```

FIX REVIEW NOTES

Fixed in PR 3894. The post-CPI source-balance read is superseded by end-of-transaction final-balance accounting; the validator no longer derives **aum_diff** from a source balance that the inner CPI has already drained.

L08: Loopscale **withdraw_strategy** and **withdraw_collateral** validators trust the caller-declared **amount** even when **withdraw_all=true** silently caps the actual transfer, letting the manager bypass the reserves invariant



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validation
s/loopscale.rs:L273-L300 [🔗](#)

RELEVANT SNIPPET

> _loopscale.rs

RUST

```

271:     Ok(Some(ValidationResult::new(aum_diff, vec![])))
272: }
273: LoopscaleInstructionKind::WithdrawStrategy => {
274:     let strategy_key =
275:         resolve_account_key(account_indexes, accounts, STRATEGY_ACCOUNT_INDEX)?;
276:     require_strategy_tracked(vault, &strategy_key)?;
277:     let lender_ta_idx = *account_indexes
278:         .get(WITHDRAW_STRATEGY_LENDER_TA_INDEX)
279:         .ok_or(ErrorCodeVaults::UnsupportedInstruction)? as usize;
280:     validate_tracked_account_index(lender_ta_idx, accounts, tracked)?;
281:     let amount = deserialize_u64_at(data, DISCRIMINATOR_LEN)?;
282:     let aum_diff = calculate_position_aum_change_for_reserve_account(
283:         vault,
284:         accounts[lender_ta_idx].key,
285:         amount,
286:         false,
287:     )?;
288:
289:     // Loopscale may auto-close the strategy on full withdrawal
290:     let strategy_account =
291:         resolve_account_info(account_indexes, accounts, STRATEGY_ACCOUNT_INDEX)?;
292:     let mutations = if is_account_closed(strategy_account) {
293:         vec![VaultMutation::UntrackLoopscaleStrategy {
294:             strategy: strategy_key,
295:         }]
296:     } else {
297:         vec![]
298:     };
299:     Ok(Some(ValidationResult::new(aum_diff, mutations)))
300: }
301: LoopscaleInstructionKind::UpdateStrategy => {
302:     let strategy_key =

```

DESCRIPTION

Two Loopscale instructions accept an **amount: u64** together with a **withdraw_all: bool** flag. When **withdraw_all** is true, Loopscale ignores the declared **amount** and moves the actual on-chain balance instead.

In the Loopscale repo, [programs/loopscale/src/instructions/strategy/withdraw.rs](#) lines 115 to 119:

RUST

```
let transfer_amount = if withdraw_all {
    TokenAccount::try_deserialize(&mut &transfer_accounts.from_ta.to_account_info().data.borrow()[..])?.amount
} else {
    amount
};
```

And [programs/loopscale/src/state/creditbook/loan.rs](#) lines 511 to 522:

RUST

```
AssetType::Spl => {
    let amount: u64 = self.collateral[collateral_index].amount.into(); // shadows
    params.amount
    if withdraw_all {
        self.collateral[collateral_index].amount = 0.into();
        withdraw_amount = amount; // uses on-chain collateral.amount
    } else {
        ...
    }
}
```

The Exponent validator handles both ix's the same way. [interaction_validations/loopscale.rs:281-296](#) (**WithdrawStrategy** arm, the **WithdrawCollateral** arm at [:206-219](#) is identical in shape):

RUST

```
LoopscaleInstructionKind::WithdrawStrategy => {
    let strategy_key =
        resolve_account_key(account_indexes, accounts, STRATEGY_ACCOUNT_INDEX)?;
    require_strategy_tracked(vault, &strategy_key)?;
    let lender_ta_idx = *account_indexes
        .get(WITHDRAW_STRATEGY_LENDER_TA_INDEX)
        .ok_or(ErrorCodeVaults::UnsupportedInstruction)? as usize;
    validate_tracked_account_index(lender_ta_idx, accounts, tracked)?;
    let amount = deserialize_u64_at(data, DISCRIMINATOR_LEN)?;
    let aum_diff = calculate_position_aum_change_for_reserve_account(
        vault,
        accounts[lender_ta_idx].key,
        amount,
        false,
    );
    ...
}
```

The validator reads **amount** (offset 8 of the inner instruction data) and never inspects the **withdraw_all** byte that follows. It passes the declared **amount** straight into **calculate_position_aum_change_for_reserve_account** with **is_increase = false**, which returns **-amount * stored_price_in_base** as the position-side delta. With **withdraw_all = true** and a large declared **amount**, the validator records a strongly negative **aum_diff** while Loopscale moves only the actual on-chain balance.

The hook's cumulative-prefix check in **validate_interaction.rs:198-212**, which calls **vault.verify_aum_change** at **vault.rs:1261-1275**, is single-sided:

RUST

```
new_in_positions = aum_in_base_in_positions + cumulative_aum_diff;
require!(new_in_positions <= effective_free_aum, ReservesExceeded);
```

There is no lower bound on **new_in_positions**. A single over-stated withdraw drags **cumulative_aum_diff** arbitrarily negative, after which subsequent inner instructions in the same Squads sync transaction can deploy reserves into other tracked positions far beyond the **reserves_share_bp** ceiling. Each subsequent positive **aum_diff** only brings the running total back toward zero; the per-step prefix check still passes because the total stays below **effective_free_aum**.

End state: real **aum_in_positions / total_aum** exceeds the configured reserves-share cap.

PROOF OF CONCEPT

Vault state: total AUM = \$1,000, **reserves_share_bp = 2000** (20%), so the designed cap is **aum_in_positions <= \$800**. Manager stages a \$1 **deposit_strategy** into a previously-registered Loopscale strategy. Post-stage: **aum_in_positions = \$1**, reserves = \$999.

In a single Squads sync transaction whose policy authorizes Loopscale program ix:

1. Inner **withdraw_strategy** with data [**disc, amount = 1e12, withdraw_all = 1**]. Loopscale moves **min(1e12, \$1) = \$1**, **strategy.token_balance** goes to 0. Validator computes **aum_diff = -1e12 * stored_price** $\boxtimes$ **-1e12**. Hook prefix check **\$1 + (-1e12) <= \$800** passes trivially.

2. Subsequent inner ix deploy all \$999 of real reserves into other tracked positions (any mix of Loopscale deposits, Kamino lend/farm/vault deposits, etc.). Each contributes a positive **aum_diff**. With **cumulative** still on the order of **-1e12**, every prefix check passes.

Post-tx: real positions $\boxtimes$ \$1,000, reserves $\boxtimes$ \$0. The vault is operating outside its reserves invariant.

The same shape works through **WithdrawCollateral**: declare a huge **amount** with **withdraw_all = true** against a tracked loan that has a single dust collateral slot. **Loan::withdraw_collateral** caps **withdraw_amount** to **collateral[idx].amount**, and the validator scales **aum_diff** by the declared amount.

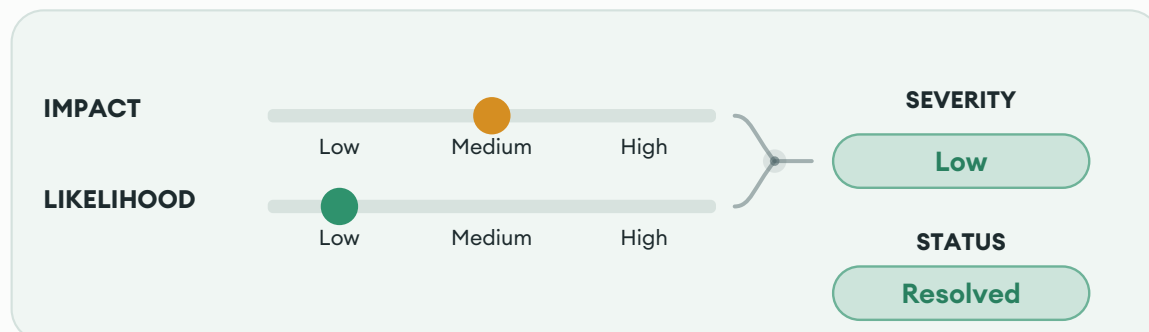
RECOMMENDATION

Reject **withdraw_all = true** in both validator arms. With **withdraw_all = false**, Loopscale uses the declared **amount** directly and reverts if it exceeds the on-chain balance, so the existing **aum_diff = -amount * stored_price** becomes correct.

FIX REVIEW NOTES

Fixed. PR 3893 initially rejected **withdraw_all = true** at the validator entry; PR 3894 then re-allowed it because the new end-of-transaction final-balance accounting handles the silent cap directly, removing the need for the entry-time ban.

L09: Kamino Vault **withdraw** validator trusts the caller-declared **shares_amount** even when Kamino silently caps to **user_shares_ata.amount**, letting the manager bypass the reserves invariant



LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/interaction_validation
s/kamino_vault.rs:L147 [🔗](#)

RELEVANT SNIPPET

>_kamino_vault.rs

RUST

```
145:     tracked: &BTreeSet<Pubkey>,  
146: ) -> Result<Option<ValidationResult>> {  
147:     let shares_amount = deserialize_u64_at(data, DISCRIMINATOR_LEN)?;  
148:     let vault_state = resolve_instruction_account(accounts, account_indexes, VAULT_STA  
TE_ACCOUNT_INDEX)?;  
149:     let shares_mint = *resolve_instruction_account(
```

DESCRIPTION

validate_withdraw reads **shares_amount** from the inner instruction's data (offset 8) and feeds it directly into the AUM-diff computation as the source-side known amount:

```
let shares_amount = deserialize_u64_at(data, DISCRIMINATOR_LEN)?;  
// ...  
let aum_diff = calculate_reserve_to_position_aum_diff_for_known_amount(  
    vault,  
    shares_ata.key,  
    destination_payout_token_account.key,  
    shares_amount, // declared value, never compared against actual balance  
    KnownAmountSide::Source,  
)?;
```

For the typical Position-to-Reserve shape (Kamino-vault-shares to tracked underlying reserve), **calculate_reserve_to_position_aum_diff_for_known_amount** (utils.rs:258-280) returns **-i64::try_from(shares_amount * stored_share_price)**.

Kamino Vault's **withdraw** handler (**kvault @ ac367c3 / handler_withdraw.rs:212**) silently caps the input to the actual on-chain shares balance:

RUST

```
let shares_amount = std::cmp::min(shares_amount, user_shares_before);
```

So the declared **shares_amount** in the instruction data can be any **u64** the manager picks. Kamino burns only **min(declared, user_shares_ata.amount)** shares and pays out only **min(...) * real_share_price** underlying. The Exponent validator never reads **user_shares_ata.amount**, never imposes an upper bound, and never compares the declared value against post-cap reality. The resulting **aum_diff** can be made arbitrarily negative.

The hook's cumulative-prefix check (**validate_interaction.rs:198-212** calling **vault.verify_aum_change** at **vault.rs:1261-1275**) is single-sided:

RUST

```
new_in_positions = aum_in_base_in_positions + cumulative_aum_diff;  
require!(new_in_positions <= effective_free_aum, ReservesExceeded); // only upper bound cap
```

There is no lower bound. A single over-stated withdraw drags **cumulative_aum_diff** arbitrarily negative, after which subsequent inner instructions in the same Squads transaction can deploy reserves beyond the **reserves_share_bp** ceiling.

PROOF OF CONCEPT

Starting state: total AUM **\$1,000**, **reserves_share_bp = 2000** (the configured minimum reserves ratio is 20%, equivalently positions are capped at 80% of total AUM = **\$800**). The vault is already at the cap: **aum_in_base_in_positions = \$800** (of which **\$1** is one Kamino-vault share at **\$1** stored share price, staged for the attack; the other **\$799** is in any mix of other tracked positions), reserves **\$200**. Normal headroom for further deploys is **\$0**.

One Squads sync transaction, two inner instructions:

#	Instruction	Real effect	Validator aum_diff	cumulative	Prefix check \$800 + cumulative <= \$800
1	Kamino Vault withdraw , data [8..16] = 1e12	min(1e12, 1) = 1 share burned, \$1 paid to reserve ATA	-1e12 * \$1 = -1e12	-1e12	\$800 - 1e12 <= \$800 passes
2	Deploy \$200 reserves into another tracked position	reserves down \$200 , positions up \$200	+\$200	-1e12 + \$200	passes

Without the bypass, the second inner instruction would fail the prefix check: **\$800 + \$200 > \$800**, **ReservesExceeded**. The over-stated negative **aum_diff** from the first inner instruction creates enough headroom to slip the second one through.

Post-hook real state: reserves around \$1, positions around \$999. The configured 80% cap on positions was \$800; the vault is now at roughly \$999, exceeding the cap by about \$199, in violation of the 20% reserve constraint.

RECOMMENDATION

A complete fix requires the validator to know the pre-interaction share balance. That value is not on-chain post-CPI, so it would have to be persisted in vault state (a **last_known_amount** on the tracked Kamino-vault-shares **TokenAccountBalance**) and advanced on every path that changes the position, both the hook and **recalculate_aum_in_base**, with attention to staleness and to double-counting across multiple burns in one transaction. That is the correct fix but a substantial one.

A smaller mitigation bounds the error instead of eliminating it, and applies to any Kamino integration that silently caps a declared amount to the available balance. The over-statement is **(declared - actual) * price**, and the cap only fires on a full burn, where **declared >= available** and the shares ATA ends at zero. The validator already reads the post-burn **user_shares_ata** balance. Restrict a full burn to a declared amount no greater than a small constant **K** (for example one share unit):

RUST

```
let declared = deserialize_u64_at(data, DISCRIMINATOR_LEN)?;
let post_shares = token_account::token_account_amount(shares_ata)?;
if post_shares == 0 {
    require!(declared <= MAX_FULL_DRAIN_DECLARED, ErrorCodeVaults::UnsupportedInstruction);
}
```

A full burn implies **available <= declared**, so with **declared <= K** the actual amount burned is also at most **K** and the over-statement is at most **K * price**: one share unit of value per instruction, economically negligible and unusable for a reserves-cap bypass. Partial burns are untouched and remain exact, because the cap cannot fire while the position retains a balance, so the declared value equals the amount burned.

The cost is a UX change for a legitimate full exit of a position holding more than **K** shares. The manager issues two inner instructions in the Squads transaction: a partial burn that takes all but a residual of at most **K**, then a second instruction that drains the residual under the **declared <= K** rule. Single partial burns are unchanged.

FIX REVIEW NOTES

Fixed in PR 3894. The team replaced the per-instruction **aum_diff** machinery with end-of-transaction final-balance accounting in the hook. Validators no longer trust caller-declared amounts, so the **shares_amount** silent-cap class can no longer occur.

5. Enhancement Opportunities

E01: Redundant manual refcount rollback before `return Err` in price-chain registration handlers

LOCATION

exponent-monorepo/fb4870a1/solana/programs/exponent_vaults/src/instructions/admin/migrate_strategy_sy_prices.rs:L157-L160 [↗](#)

RELEVANT SNIPPET

```
> _migrate_strategy_sy_prices.rs RUST
155: drop(exponent_prices);
156:
157: if let Err(err) = write_vault_data(&ctx.accounts.vault.to_account_info(), &serialized)
158: {
159:     let mut exponent_prices = load_exponent_prices_mut(&ctx.accounts.exponent_prices)?
160: ;
161:     release_validated_prices(&mut exponent_prices, &validated_prices);
162:     return Err(err);
163: }
```

DESCRIPTION

When a strategy position is registered, its price chains are validated by `Priceld::validate_price`. On success, that call increments a usage refcount on every price entry the chain references (via `add_price_pointers`); the refcount prevents those entries from being removed while a position still uses them. The matching `release_price` decrements it.

`add_orderbook_entry`, `add_yield_position_entry`, and `add_clmm_position_entry` (in `manage_vault_settings.rs`) register positions that carry two price chains, a PT chain and an SY chain. They validate the PT chain first, then the SY chain. If PT succeeds but SY fails, they manually call `release_price` on the PT chain to undo its refcount increment before returning the error:

```
RUST
entry.price_id_pt.validate_price(exponent_prices, core_vault.mint_pt, Some(entry.base_mint), vault.underlying_mint)?;

if let Err(err) = entry.price_id_sy.validate_price(exponent_prices, core_vault.mint_sy, Some(entry.base_mint), vault.underlying_mint) {
    entry.price_id_pt.release_price(exponent_prices)?; // redundant
    return Err(err);
}
```

(`migrate_strategy_sy_prices` does the same thing over a list of chains via the `release_validated_prices` helper.)

This manual rollback has no effect. The refcount lives in the `exponent_prices` account. The branch ends in `return Err`, which fails the whole instruction, and Solana then restores every account the instruction touched (including `exponent_prices`) to its state before the instruction. The PT refcount

increment is reverted by the runtime whether or not `release_price` ran first, so the manual call only changes state that is about to be discarded. These error paths are the only callers of `release_price` / `release_validated_prices` in these handlers.

POTENTIAL BENEFIT

Saves the refcount-decrement CU on failed registrations. No behavior change.

RECOMMENDATION

Drop the manual `release_price` / `release_validated_prices` call on each error path; keep the explicit `if let Err { return Err }`.

FIX REVIEW NOTES

Implemented in PR 3894. The redundant manual refcount rollback before `return Err` was removed.

E02: Validating the TWAP window size against the minimum the stored-price path requires will fail misconfiguration early instead of as a vault-wide recalc revert

LOCATION

exponent-monorepo/fb4870a1/solana/libraries/exponent_vaults_integrations/src/price_sources/twap.rs:L35-L71 [↗](#)

RELEVANT SNIPPET

>_twap.rs

RUST

```
33:
34: impl PriceInterfaceAccounts {
35:     pub fn new_twap(
36:         interface_accounts: Vec<Pubkey>,
37:         source_price_type: PriceType,
38:         twap_window_size: u8,
39:         update_interval_seconds: u32,
40:         max_delta_bps_per_update: u32,
41:     ) -> Result<Self> {
42:         require!(
43:             !matches!(source_price_type, PriceType::Twap | PriceType::Yt),
44:             ErrorCodeVaults::InvalidPriceType
45:         );
46:         require!(
47:             twap_window_size > 0,
48:             ErrorCodeVaults::InvalidTwapUpdateInterval
49:         );
50:         require!(
51:             update_interval_seconds > 0,
52:             ErrorCodeVaults::InvalidTwapUpdateInterval
53:         );
54:         require!(
55:             max_delta_bps_per_update > 0,
56:             ErrorCodeVaults::InvalidTwapApyDeltaLimit
57:         );
58:
59:         Ok(Self {
60:             interface_accounts,
61:             state: PriceInterfaceState::Twap(TwapState {
62:                 config: TwapConfig {
63:                     source_price_type,
64:                     window_size: twap_window_size,
65:                     update_interval_seconds,
66:                     max_delta_bps_per_update,
67:                 },
68:                 observations: Vec::new(),
69:             }),
70:         })
71:     }
72:
73:     pub fn twap_source_price_type(&self) -> Result<PriceType> {
```

DESCRIPTION

`new_twap` validates `twap_window_size > 0`. The consumer side, `read_stored_price`, requires at least `STORED_TWAP_MIN_RECENT_OBSERVATIONS` (4) recent observations before it returns a price. A TWAP created with a window size of 1, 2, or 3 is accepted at creation but can never accumulate 4 observations, so `calculate_stored_price` always fails for it. Any tracked position whose price chain references that entry then makes `recalculate_aum_in_base` revert, blocking deposits and withdrawals for the vault until the configuration is corrected.

The degenerate window size is accepted silently and only surfaces later, as a vault-wide recalculation revert that an operator has to trace back to the price configuration, rather than being rejected at the point it is set.

POTENTIAL BENEFIT

Turns a silently-accepted, permanently-unusable price configuration (whose only symptom is a later vault-wide recalc revert) into an immediate, localized failure at the time the TWAP is added, with a clear error.

RECOMMENDATION

Tighten the window-size check in `new_twap` to the minimum the stored-price path requires, reusing the existing constant so the producer and consumer bounds cannot drift apart:

RUST

```
require!(
    twap_window_size as usize >= STORED_TWAP_MIN_RECENT_OBSERVATIONS,
    ErrorCodeVaults::InvalidTwapUpdateInterval
);
```

This replaces the current `twap_window_size > 0` check.

FIX REVIEW NOTES

Implemented in PR 3899. TWAP window-size validation now requires the window size to be at least the stored-price minimum observation count of 4.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.